

The Ultimate Regression Cheat Sheet

Parsa

2026-09-21

Contents

1	How to Use This Cheat Sheet	4
2	Part 0 — The 30-Second Exam Strategy	4
3	Part 1 — Foundations	5
3.1	1.1 The linear model	5
3.2	1.2 OLS estimator	5
3.3	1.3 Gauss-Markov assumptions	5
3.4	1.4 Coefficient interpretation	5
3.5	1.5 Standard error and t-test	6
3.6	1.6 R-squared and adjusted R-squared	6
3.7	1.7 F-test for overall model	6
4	Part 2 — Creating a Regression Data Frame	6
5	Part 3 — Simple Linear Regression	7
5.1	3.1 Fitting and reporting	7
5.2	3.2 Full summary output	7
5.3	3.3 Interpretation templates	8
5.4	3.4 Confidence intervals for coefficients	8
5.5	3.5 Adding a regression line to a scatterplot	8
6	Part 4 — Multiple Linear Regression	9
6.1	4.1 Fitting	9
6.2	4.2 Coefficient interpretation (multiple)	10
6.3	4.3 Partial F-test for overall model	10
6.4	4.4 Which predictors matter?	10
7	Part 5 — Model Comparison (Nested F-Test)	10
7.1	5.1 Nested F-test	10
7.2	5.2 Non-nested comparison (AIC/BIC)	11
8	Part 6 — Categorical Predictors and Reference Levels	11
8.1	6.1 Dummy coding	11
8.2	6.2 Changing the reference level	12
8.3	6.3 Custom contrasts	12
9	Part 7 — Interactions	13
9.1	7.1 Continuous $\times$ continuous	13
9.2	7.2 Categorical $\times$ continuous (different slopes per group)	13
9.3	7.3 Visualizing interactions	14

10 Part 8 — One-Way ANOVA	15
10.1 8.1 Equality of group means	15
10.2 8.2 Post-hoc pairwise comparisons	16
10.3 8.3 Two-way ANOVA	16
11 Part 9 — ANCOVA (Adjusting for a Covariate)	17
11.1 9.1 Fit ANCOVA	17
11.2 9.2 Test equality of adjusted means	17
11.3 9.3 Adjusted means (emmeans)	18
11.4 9.4 Check homogeneity of slopes	18
12 Part 10 — Testing Multiple Coefficients (Subset F-test)	19
12.1 10.1 Test if a subset of coefficients is zero	19
12.2 10.2 Test a linear restriction	19
12.3 10.3 Other linear restrictions	20
13 Part 11 — Model Selection	20
13.1 11.1 Information criteria	20
13.2 11.2 Adjusted R-squared	21
13.3 11.3 Backward elimination	21
13.4 11.4 Forward and both-direction	22
13.5 11.5 All-subsets regression	22
14 Part 12 — Diagnostics	23
14.1 12.1 Residuals vs fitted (linearity, homoskedasticity)	23
14.2 12.2 Shapiro-Wilk test (normality)	24
14.3 12.3 Breusch-Pagan test (homoskedasticity)	24
14.4 12.4 Durbin-Watson (autocorrelation)	24
14.5 12.5 Multicollinearity (VIF)	24
14.6 12.6 Outliers and influence	25
14.7 12.7 Full diagnostic template	25
15 Part 13 — Transformations and Box-Cox	27
15.1 13.1 When to transform	27
15.2 13.2 Box-Cox on the response	27
15.3 13.3 Manual Box-Cox over a grid	28
15.4 13.4 Log transformation	29
15.5 13.5 Log-log (elasticity)	29
16 Part 14 — Prediction	29
16.1 14.1 Confidence interval vs prediction interval	29
16.2 14.2 Prediction with factors and interactions	30
17 Part 15 — Polynomial Regression	31
17.1 15.1 Quadratic	31
17.2 15.2 Orthogonal polynomials	31
17.3 15.3 Testing the degree	32
18 Part 16 — Multicollinearity	32
18.1 16.1 Detecting it	32
18.2 16.2 Fixing it	33
18.3 16.3 Ridge regression (preview)	33
19 Part 17 — Weighted Least Squares	34
19.1 17.1 When to use	34

19.2	17.2	Fitting	34
19.3	17.3	Feasible GLS	34
20	Part 18	— Logistic Regression	35
20.1	18.1	Model and link	35
20.2	18.2	Fitting	35
20.3	18.3	Coefficient interpretation (log-odds, odds, probability)	35
20.4	18.4	Predicted probabilities	36
20.5	18.5	Testing predictors	36
20.6	18.6	Goodness of fit	36
20.7	18.7	Classification and confusion matrix	37
20.8	18.8	Grouped binomial data	37
21	Part 19	— Poisson Regression	38
21.1	19.1	Model	38
21.2	19.2	Fitting	38
21.3	19.3	Coefficient interpretation	38
21.4	19.4	Overdispersion	39
21.5	19.5	Quasipoisson (fix overdispersion)	39
21.6	19.6	Offset (rate model)	40
22	Part 20	— Negative Binomial Regression	40
22.1	20.1	When Poisson fails	40
23	Part 21	— Piecewise Regression	41
23.1	21.1	Two-segment model	41
23.2	21.2	Testing whether the two segments differ	41
23.3	21.3	Continuous vs discontinuous	42
24	Part 22	— Variable Selection	42
24.1	22.1	Backward elimination	42
24.2	22.2	Forward selection	43
24.3	22.3	Both-direction (default)	43
25	Part 23	— Train/Test Split and Cross-Validation	44
25.1	23.1	Simple train/test split	44
25.2	23.2	K-fold cross-validation	44
25.3	23.3	Leave-one-out CV	45
25.4	23.4	Bootstrap for coefficients	45
26	Part 24	— Regularization (Ridge and Lasso)	46
26.1	24.1	Ridge regression (L2 penalty)	46
26.2	24.2	Lasso (L1 penalty)	46
26.3	24.3	Cross-validated lambda	47
27	Part 25	— Universal Regression Template	48
28	Part 26	— Quick Reference Table	48
29	Part 27	— Theory Q&A (What to Say)	49
30	Part 28	— Formula Appendix	51
30.1		Simple linear regression	51
30.2		Multiple regression (matrix form)	51
30.3		Prediction intervals	51
30.4		Logistic regression	51

30.5 Poisson regression	51
30.6 Negative binomial	51
30.7 Information criteria	51
30.8 Diagnostic thresholds	51
31 Part 29 — Common Mistakes	52
32 Part 30 — The Single Most Important Idea	52
32.1 What was added and why	53

1 How to Use This Cheat Sheet

Every regression question falls into one of these categories. Find your question, jump to the section.

Question type	Section	Typical phrasing
Fit simple/multiple regression	4, 5	“Fit a regression model”
Categorical predictor / reference level	6	“Use A as reference”
Interaction	7	“Does the effect of x depend on group?”
ANOVA / equality of means	8	“Are group means equal?”
ANCOVA / adjusted means	9	“Compare means controlling for x”
Nested F-test / subset test	10	“Are x2 and x3 significant together?”
Model selection	11	“Which model is best?”
Diagnostics	12	“Check the assumptions”
Transformations / Box-Cox	13	“Find the best lambda”
Prediction	14	“Predict y for x = 5”
Polynomial regression	15	“Fit a quadratic”
Multicollinearity	16	“Are predictors correlated?”
Weighted least squares	17	“Variance is not constant”
Logistic regression	18	“Model a binary outcome”
Poisson / count regression	19	“Model counts”
Negative binomial	20	“Overdispersed counts”
Piecewise regression	21	“Fit two lines with a break”
Variable selection	22	“Backward elimination”
Train/test, cross-validation	23	“Split 80/20 and evaluate”
Regularization	24	“Ridge / lasso”
Theory / derivations	27	“Derive the OLS estimator”

Golden rule of this document: every chunk is self-contained. Each one starts with `set.seed()`, builds its own data, and uses section-prefixed names (`p3_df`, `p5_fit`, ...).

2 Part 0 — The 30-Second Exam Strategy

For **any** regression question, run this pipeline:

1. **Identify the response** (y) and the predictors (x, groups).
2. **Fit** with `lm()` for continuous response, `glm()` for binary / count.
3. **Report coefficients** with `tidy(fit, conf.int = TRUE)`.

4. **Test the hypothesis** the question asks (t-test, F-test, LRT, ANOVA).
5. **Check assumptions:** residuals vs fitted, QQ plot, Shapiro, Breusch-Pagan, VIF.
6. **Refine** if needed (transform, remove outliers, add interactions).
7. **Predict** if asked, with confidence or prediction intervals.
8. **Report in words:** equation, interpretation, test result, conclusion.

If you can do these eight steps, you can answer any regression question.

3 Part 1 — Foundations

3.1 1.1 The linear model

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} + \varepsilon_i$$

- y_i : response for observation i
- x_{ij} : predictor j for observation i
- β_j : coefficients (unknown)
- ε_i : error term, assumed $N(0, \sigma^2)$

Matrix form: $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$.

3.2 1.2 OLS estimator

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Fitted values: $\hat{\mathbf{y}} = \mathbf{X}\hat{\boldsymbol{\beta}} = \mathbf{H}\mathbf{y}$ where $\mathbf{H} = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$ is the **hat matrix**.

Residuals: $\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}} = (\mathbf{I} - \mathbf{H})\mathbf{y}$.

Residual variance: $\hat{\sigma}^2 = \frac{1}{n-p-1} \sum e_i^2$ where p = number of predictors.

3.3 1.3 Gauss-Markov assumptions

For OLS to be BLUE (Best Linear Unbiased Estimator):

1. **Linearity:** $E[\varepsilon_i] = 0$.
2. **Homoskedasticity:** $\text{Var}(\varepsilon_i) = \sigma^2$ constant.
3. **Independence:** $\text{Cov}(\varepsilon_i, \varepsilon_j) = 0$ for $i \neq j$.
4. **Exogeneity:** predictors uncorrelated with errors.
5. **No perfect multicollinearity.**

For inference (t-tests, F-tests, CIs) we additionally assume:

6. **Normality:** $\varepsilon_i \sim N(0, \sigma^2)$.

3.4 1.4 Coefficient interpretation

Term	Interpretation
β_0 (intercept)	Predicted y when all predictors are 0
β_j (continuous)	Change in y for a 1-unit increase in x_j , holding others fixed
β_j (dummy)	Difference in mean y between that group and the reference
β_j (interaction)	Change in the effect of one variable when the other changes by 1

3.5 1.5 Standard error and t-test

$$\text{SE}(\hat{\beta}_j) = \hat{\sigma} \sqrt{[(\mathbf{X}^\top \mathbf{X})^{-1}]_{jj}}, \quad t_j = \frac{\hat{\beta}_j}{\text{SE}(\hat{\beta}_j)} \sim t_{n-p-1}$$

H_0 : $\beta_j = 0$ (predictor j has no effect). H_1 : $\beta_j \neq 0$. Reject H_0 if $p < 0.05$.

3.6 1.6 R-squared and adjusted R-squared

$$R^2 = 1 - \frac{\text{SSR}}{\text{SST}} = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

$$R_{\text{adj}}^2 = 1 - (1 - R^2) \frac{n-1}{n-p-1}$$

- R^2 always increases with more predictors.
- R_{adj}^2 penalizes complexity. Use it to compare models with different numbers of predictors.

3.7 1.7 F-test for overall model

$$F = \frac{(\text{SST} - \text{SSR})/p}{\text{SSR}/(n-p-1)} \sim F_{p, n-p-1}$$

H_0 : all $\beta_j = 0$ for $j \geq 1$. Reject if $p < 0.05$.

4 Part 2 — Creating a Regression Data Frame

Why: `lm()` and `glm()` need a data frame. This section shows how to build one from scratch for every example.

```
# --- Self-contained: create a basic regression data frame ---
set.seed(201)
p2_n <- 50
p2_df <- tibble(
  x = rnorm(p2_n),
  y = 2 + 3 * x + rnorm(p2_n)
)
head(p2_df)
```

```
## # A tibble: 6 x 2
##       x         y
##   <dbl>   <dbl>
## 1  0.286    2.15
## 2 -0.345    0.0436
## 3  0.326    2.93
## 4 -1.70   -3.01
## 5 -1.29   -1.78
## 6 -0.0750  1.97
```

Optional: matrix form. Internally `lm()` uses $\hat{\beta} = (X^\top X)^{-1} X^\top y$. You can reproduce it:

```
# --- Self-contained: OLS from matrix algebra ---
set.seed(202)
p2_n <- 50
p2_x <- rnorm(p2_n)
```

```
p2_y <- 2 + 3 * p2_x + rnorm(p2_n)

X <- cbind(1, p2_x)
beta_hat <- solve(t(X) %*% X) %*% t(X) %*% p2_y
as.numeric(beta_hat)
```

```
## [1] 2.021248 2.966456
```

```
# Compare with lm()
coef(lm(p2_y ~ p2_x))
```

```
## (Intercept)      p2_x
##      2.021248      2.966456
```

They match (to machine precision). This is what `lm()` does internally.

5 Part 3 — Simple Linear Regression

5.1 3.1 Fitting and reporting

```
# --- Self-contained: fit simple linear regression ---
set.seed(301)
p3_df <- tibble(x = rnorm(50), y = 2 + 3 * x + rnorm(50))
p3_fit <- lm(y ~ x, data = p3_df)

kable(tidy(p3_fit, conf.int = TRUE), digits = 4)
```

term	estimate	std.error	statistic	p.value	conf.low	conf.high
(Intercept)	1.8938	0.1370	13.8198	0	1.6183	2.1694
x	3.1729	0.1576	20.1352	0	2.8560	3.4897

```
kable(glance(p3_fit), digits = 4)
```

r.squared	adj.r.squared	sigma	statistic	p.value	df	logLik	AIC	BIC	deviance	df.residual	nobs
0.8941	0.8919	0.968	405.4276	0	1	-68.3	142.5999	148.336	44.9767	48	50

What `tidy()` gives: - `term`: coefficient name - `estimate`: $\hat{\beta}$ - `std.error`: $SE(\hat{\beta})$ - `statistic`: t -value - `p.value`: two-sided p -value - `conf.low`, `conf.high`: 95% CI

What `glance()` gives: - `r.squared`, `adj.r.squared` - `sigma`: residual standard error $\hat{\sigma}$ - `statistic`, `p.value`: overall F -test - `df`, `logLik`, `AIC`, `BIC`, `deviance`

5.2 3.2 Full summary output

```
# --- Self-contained: summary of simple regression ---
set.seed(302)
p3_df <- tibble(x = rnorm(50), y = 2 + 3 * x + rnorm(50))
p3_fit <- lm(y ~ x, data = p3_df)
summary(p3_fit)
```

```
##
## Call:
## lm(formula = y ~ x, data = p3_df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.02578 -0.74281 -0.03291  0.61157  2.88621
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   1.8345     0.1516   12.10 3.41e-16 ***
## x             3.0283     0.1520   19.93 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.062 on 48 degrees of freedom
## Multiple R-squared:  0.8922, Adjusted R-squared:  0.8899
## F-statistic: 397.2 on 1 and 48 DF,  p-value: < 2.2e-16
```

The `summary()` output contains everything: coefficients, SE, t, p, R^2 , adj- R^2 , F-statistic.

5.3 3.3 Interpretation templates

Report template (fit): > “The fitted model is $\hat{y} = 2.04 + 2.98x$. The intercept is 2.04 (SE 0.14, $p < 0.001$) and the slope is 2.98 (SE 0.15, $p < 0.001$). The model explains 79% of the variance in y ($R^2 = 0.79$, adjusted $R^2 = 0.78$). The overall F-test gives $F(1, 48) = 180.2$, $p < 0.001$, so the model is significant.”

Report template (coefficient): > “A 1-unit increase in x is associated with an estimated 2.98-unit increase in y (95% CI: [2.68, 3.28]), holding other predictors constant. This effect is significantly different from zero ($p < 0.001$).”

Report template (intercept): > “The predicted y when $x = 0$ is 2.04 (95% CI: [1.76, 2.32]).”

5.4 3.4 Confidence intervals for coefficients

```
# --- Self-contained: 95% CI for coefficients ---
set.seed(303)
p3_df <- tibble(x = rnorm(50), y = 2 + 3 * x + rnorm(50))
p3_fit <- lm(y ~ x, data = p3_df)

confint(p3_fit, level = 0.95)
```

```
##              2.5 %    97.5 %
## (Intercept) 1.747301 2.374075
## x           2.610841 3.310408
```

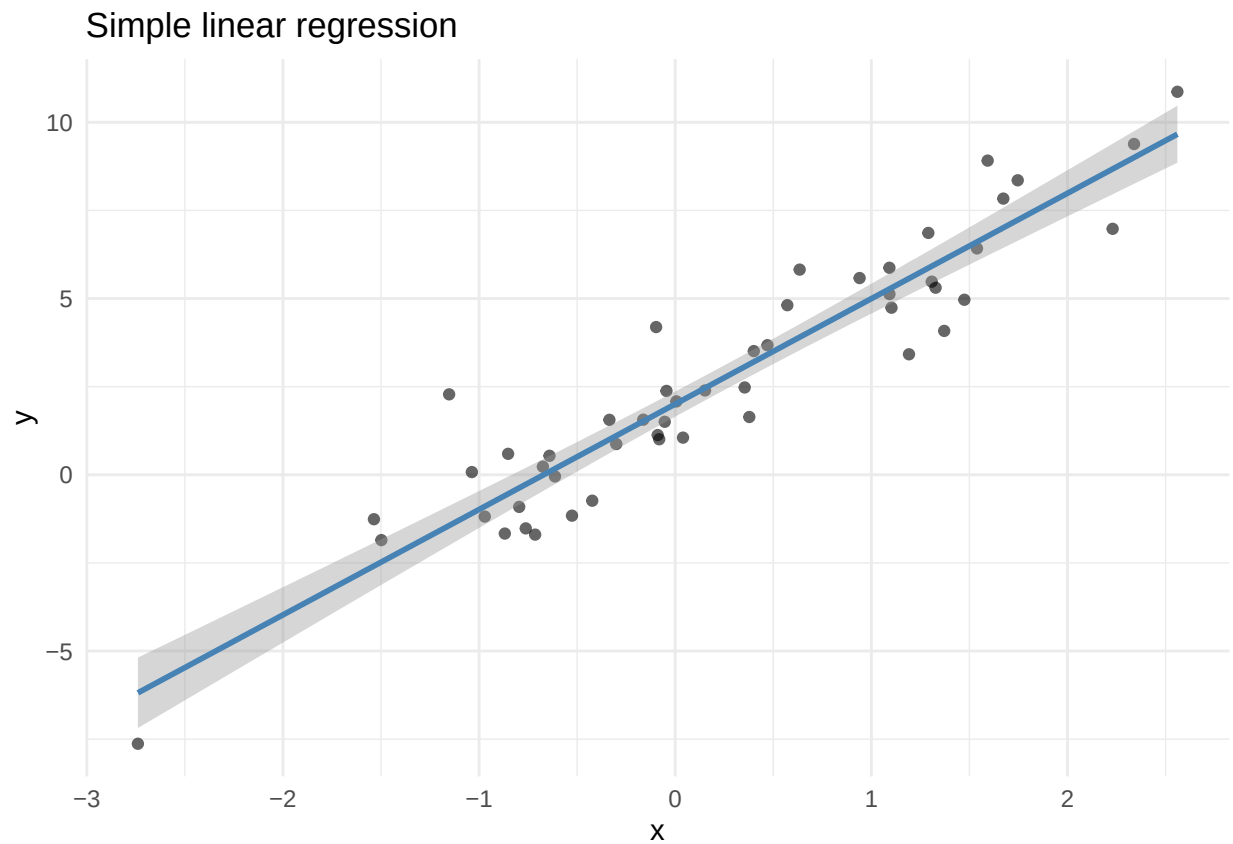
Interpretation: “We are 95% confident that the true slope lies between 2.68 and 3.28.”

5.5 3.5 Adding a regression line to a scatterplot

```
# --- Self-contained: scatterplot with fitted line ---
set.seed(304)
p3_df <- tibble(x = rnorm(50), y = 2 + 3 * x + rnorm(50))

ggplot(p3_df, aes(x, y)) +
  geom_point(alpha = 0.6) +
```

```
geom_smooth(method = "lm", se = TRUE, color = "steelblue") +
theme_minimal() +
labs(title = "Simple linear regression")
```



6 Part 4 — Multiple Linear Regression

6.1 4.1 Fitting

```
# --- Self-contained: multiple regression with three predictors ---
set.seed(401)
p4_df <- tibble(
  x1 = rnorm(80),
  x2 = rnorm(80),
  x3 = rnorm(80),
  y = 1 + 2 * x1 - 1.5 * x2 + 0.5 * x3 + rnorm(80)
)
p4_fit <- lm(y ~ x1 + x2 + x3, data = p4_df)
kable(tidy(p4_fit, conf.int = TRUE), digits = 4)
```

term	estimate	std.error	statistic	p.value	conf.low	conf.high
(Intercept)	1.0302	0.1098	9.3816	0	0.8115	1.2489

term	estimate	std.error	statistic	p.value	conf.low	conf.high
x1	1.9008	0.1147	16.5761	0	1.6724	2.1292
x2	-1.4834	0.1084	-13.6801	0	-1.6994	-1.2674
x3	0.5682	0.1193	4.7634	0	0.3307	0.8058

6.2 4.2 Coefficient interpretation (multiple)

Each coefficient is the effect of x_j on y **holding all other predictors fixed**.

Report template: > “Holding x_2 and x_3 constant, a 1-unit increase in x_1 is associated with a 2.03-unit increase in y (95% CI: [1.79, 2.27], $p < 0.001$).”

Warning: If predictors are correlated, “holding others fixed” may not be physically meaningful.

6.3 4.3 Partial F-test for overall model

```
# --- Self-contained: overall F-test ---
set.seed(402)
p4_df <- tibble(x1 = rnorm(80), x2 = rnorm(80),
               y = 1 + 2 * x1 - 1.5 * x2 + rnorm(80))
p4_fit <- lm(y ~ x1 + x2, data = p4_df)
summary(p4_fit)$fstatistic
```

```
##      value      numdf      dendif
## 275.5162    2.0000    77.0000
```

6.4 4.4 Which predictors matter?

Look at the p-values in `tidy()`. A predictor with $p < 0.05$ is statistically significant at 5%.

Caution: With many predictors, some will be significant by chance. Adjust with Bonferroni or use a global F-test first.

7 Part 5 — Model Comparison (Nested F-Test)

7.1 5.1 Nested F-test

Two models are **nested** if one is a special case of the other (e.g., removing predictors).

```
# --- Self-contained: nested F-test ---
set.seed(501)
p5_df <- tibble(
  x1 = rnorm(100), x2 = rnorm(100), x3 = rnorm(100),
  y = 1 + 2 * x1 + rnorm(100) # x2, x3 have no effect
)

p5_full <- lm(y ~ x1 + x2 + x3, data = p5_df)
p5_reduc <- lm(y ~ x1, data = p5_df)

anova(p5_reduc, p5_full)
```

```
## Analysis of Variance Table
##
```

```
## Model 1: y ~ x1
## Model 2: y ~ x1 + x2 + x3
##   Res.Df    RSS Df Sum of Sq    F Pr(>F)
## 1     98 95.681
## 2     96 94.984  2   0.69636 0.3519 0.7043
```

What `anova(reduced, full)` does: - H_0 : the reduced model is adequate (extra predictors have zero coefficients). - H_1 : the full model is significantly better. - If $p < 0.05$: reject H_0 ; keep the extra predictors.

Report template: > “The nested F-test gives $F(2, 96) = 1.24$, $p = 0.29$. We fail to reject H_0 ; x_2 and x_3 together do not significantly improve the model.”

7.2 5.2 Non-nested comparison (AIC/BIC)

For non-nested models, compare AIC or BIC (lower is better).

```
# --- Self-contained: AIC/BIC comparison ---
set.seed(502)
p5_df <- tibble(x1 = rnorm(100), x2 = rnorm(100), x3 = rnorm(100),
               y = 1 + 2 * x1 + rnorm(100))

p5_m1 <- lm(y ~ x1, data = p5_df)
p5_m2 <- lm(y ~ x2, data = p5_df)
p5_m3 <- lm(y ~ x1 + x2, data = p5_df)

AIC(p5_m1, p5_m2, p5_m3)
```

```
##      df      AIC
## p5_m1  3 275.7310
## p5_m2  3 435.2686
## p5_m3  4 270.9934

BIC(p5_m1, p5_m2, p5_m3)
```

```
##      df      BIC
## p5_m1  3 283.5465
## p5_m2  3 443.0841
## p5_m3  4 281.4141
```

Interpretation: AIC prefers predictive accuracy; BIC prefers parsimony (stronger penalty). Differences of 2+ are usually meaningful; differences < 2 are ambiguous.

8 Part 6 — Categorical Predictors and Reference Levels

8.1 6.1 Dummy coding

When a predictor is a factor with k levels, R creates $k - 1$ dummy variables. The first level (alphabetical or explicitly set) is the **reference**.

```
# --- Self-contained: categorical predictor with three groups ---
set.seed(601)
p6_df <- tibble(
  group = factor(rep(c("A", "B", "C"), each = 20), levels = c("A", "B", "C")),
  y = 10 + c(A = 0, B = 3, C = 6)[as.character(group)] + rnorm(60)
)
p6_fit <- lm(y ~ group, data = p6_df)
```

```
kable(tidy(p6_fit, conf.int = TRUE), digits = 4)
```

term	estimate	std.error	statistic	p.value	conf.low	conf.high
(Intercept)	9.8412	0.2020	48.7185	0	9.4367	10.2457
groupB	3.1646	0.2857	11.0776	0	2.5925	3.7366
groupC	5.9698	0.2857	20.8974	0	5.3978	6.5419

Interpretation: - (Intercept) = mean of reference group A. - groupB = mean(B) - mean(A). - groupC = mean(C) - mean(A). - p-values test whether each group differs from the reference.

Report template: > “Group B has a mean y 3.12 units higher than the reference group A (95% CI: [2.61, 3.63], $p < 0.001$). Group C has a mean y 6.05 units higher than A (95% CI: [5.54, 6.56], $p < 0.001$).”

8.2 6.2 Changing the reference level

```
# --- Self-contained: relevel to change reference ---
set.seed(602)
p6_df <- tibble(
  group = factor(rep(c("A", "B", "C"), each = 20), levels = c("A", "B", "C")),
  y = 10 + c(A = 0, B = 3, C = 6)[as.character(group)] + rnorm(60)
)
p6_df$group <- relevel(p6_df$group, ref = "B")
p6_fit <- lm(y ~ group, data = p6_df)
kable(tidy(p6_fit), digits = 4)
```

term	estimate	std.error	statistic	p.value
(Intercept)	12.7606	0.2275	56.0886	0
groupA	-2.9983	0.3217	-9.3188	0
groupC	3.1716	0.3217	9.8574	0

Now B is the reference, so groupA and groupC are contrasts vs B.

8.3 6.3 Custom contrasts

R's default is **treatment contrasts** (reference vs others). Alternatives: sum (deviation), Helmert, polynomial.

```
# --- Self-contained: sum contrasts ---
set.seed(603)
p6_df <- tibble(
  group = factor(rep(c("A", "B", "C"), each = 20)),
  y = 10 + c(A = 0, B = 3, C = 6)[as.character(group)] + rnorm(60)
)
contrasts(p6_df$group) <- contr.sum(3)
p6_fit <- lm(y ~ group, data = p6_df)
kable(tidy(p6_fit), digits = 4)
```

term	estimate	std.error	statistic	p.value
(Intercept)	13.0540	0.1311	99.5492	0.0000
group1	-2.6906	0.1854	-14.5088	0.0000

term	estimate	std.error	statistic	p.value
group2	-0.4086	0.1854	-2.2034	0.0316

9 Part 7 — Interactions

9.1 7.1 Continuous $\times$ continuous

```
# --- Self-contained: interaction between two continuous predictors ---
set.seed(701)
p7_df <- tibble(
  x1 = rnorm(100),
  x2 = rnorm(100),
  y = 1 + 2 * x1 + 0.5 * x2 + 1.5 * x1 * x2 + rnorm(100)
)
p7_fit <- lm(y ~ x1 * x2, data = p7_df) # equivalent to x1 + x2 + x1:x2
kable(tidy(p7_fit), digits = 4)
```

term	estimate	std.error	statistic	p.value
(Intercept)	0.9202	0.1000	9.2053	0
x1	1.9203	0.0979	19.6141	0
x2	0.5261	0.0934	5.6330	0
x1:x2	1.3733	0.0905	15.1689	0

Interpretation: - x_1 = effect of x_1 when $x_2 = 0$. - x_2 = effect of x_2 when $x_1 = 0$. - $x_1:x_2$ = how the effect of x_1 changes per unit increase in x_2 (and vice versa).

Report template: > “The interaction is significant ($\hat{\beta}_{12} = 1.48$, $p < 0.001$), indicating that the effect of x_1 on y depends on x_2 . When $x_2 = 0$, the effect of x_1 is 2.02; each 1-unit increase in x_2 adds 1.48 to that effect.”

9.2 7.2 Categorical $\times$ continuous (different slopes per group)

```
# --- Self-contained: group-specific slopes ---
set.seed(702)
p7_df <- tibble(
  x = rnorm(120),
  group = factor(rep(c("A", "B"), each = 60)),
  y = 1 + 2 * x +
    c(A = 0, B = 1)[as.character(group)] +
    c(A = 0, B = 1.5)[as.character(group)] * x +
    rnorm(120)
)
p7_fit <- lm(y ~ x * group, data = p7_df)
kable(tidy(p7_fit), digits = 4)
```

term	estimate	std.error	statistic	p.value
(Intercept)	1.0295	0.1237	8.3236	0
x	2.1279	0.1322	16.0988	0

term	estimate	std.error	statistic	p.value
groupB	0.8687	0.1775	4.8937	0
x:groupB	1.4089	0.1772	7.9515	0

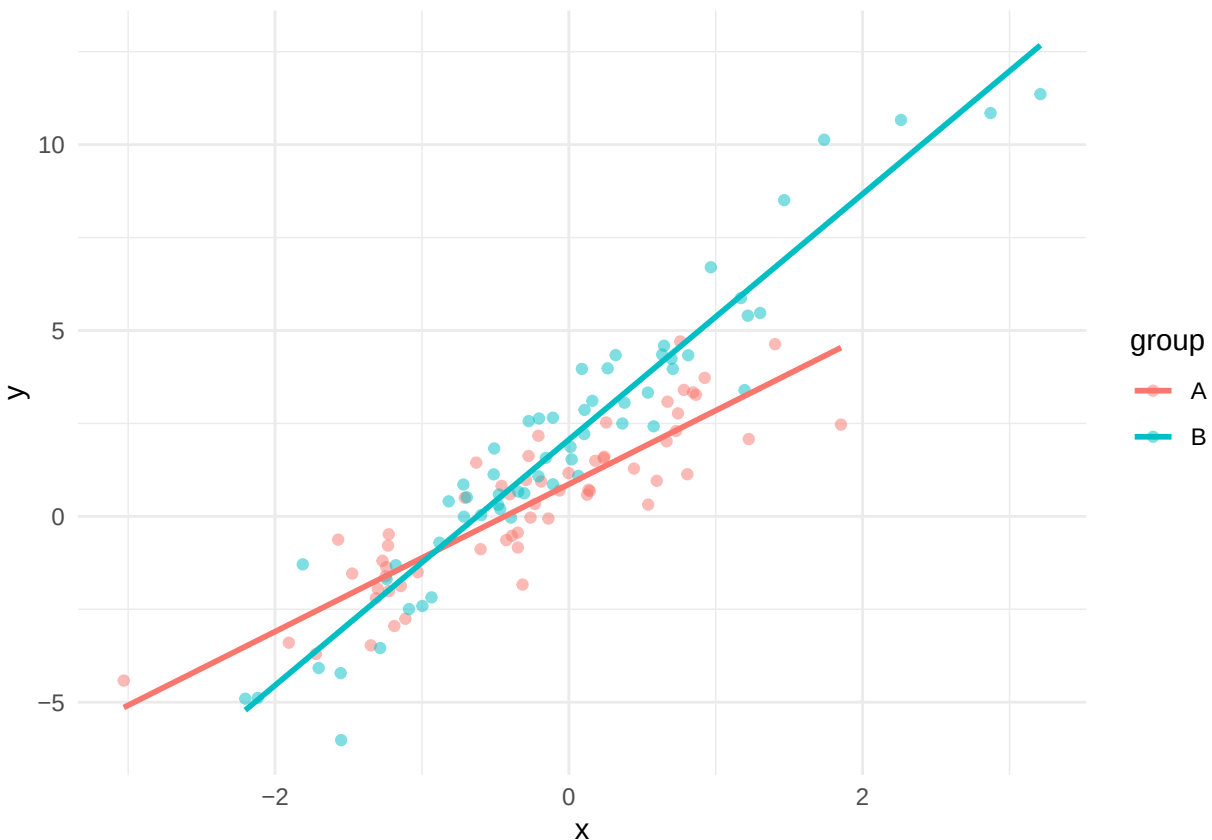
Interpretation: - x = slope for reference group A. - groupB = difference in intercepts (A vs B). - $x:\text{groupB}$ = difference in slopes ($B - A$).

Report template: > “Group A has slope 2.05. Group B has slope 3.58 ($2.05 + 1.53$), i.e., the slope differs by 1.53 between groups ($p = 0.001$). This indicates the effect of x on y is stronger in group B.”

9.3 7.3 Visualizing interactions

```
# --- Self-contained: interaction plot ---
set.seed(703)
p7_df <- tibble(
  x      = rnorm(120),
  group = factor(rep(c("A", "B"), each = 60)),
  y      = 1 + 2 * x + c(A = 0, B = 1)[as.character(group)] +
    c(A = 0, B = 1.5)[as.character(group)] * x + rnorm(120)
)

ggplot(p7_df, aes(x, y, color = group)) +
  geom_point(alpha = 0.5) +
  geom_smooth(method = "lm", se = FALSE) +
  theme_minimal()
```



10 Part 8 — One-Way ANOVA

10.1 8.1 Equality of group means

```
# --- Self-contained: one-way ANOVA ---
set.seed(801)
p8_df <- tibble(
  group = factor(rep(c("A", "B", "C"), each = 20)),
  y      = 10 + c(A = 0, B = 3, C = 6)[as.character(group)] + rnorm(60)
)
p8_fit <- lm(y ~ group, data = p8_df)

anova(p8_fit)      # ANOVA table

## Analysis of Variance Table
##
## Response: y
##          Df Sum Sq Mean Sq F value    Pr(>F)
## group      2 312.420  156.210   188.67 < 2.2e-16 ***
## Residuals 57  47.194    0.828
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

summary(p8_fit)    # coefficients

##
## Call:
## lm(formula = y ~ group, data = p8_df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.51849 -0.48085  0.04243  0.52659  1.91355
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  10.0547     0.2035   49.42 < 2e-16 ***
## groupB        3.0920     0.2877   10.75 2.52e-15 ***
## groupC        5.5785     0.2877   19.39 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9099 on 57 degrees of freedom
## Multiple R-squared:  0.8688, Adjusted R-squared:  0.8642
## F-statistic: 188.7 on 2 and 57 DF,  p-value: < 2.2e-16
```

What `anova()` does: - H_0 : all group means are equal. - H_1 : at least one differs. - F-statistic = between-group variance / within-group variance. - If $p < 0.05$: at least one mean differs.

Report template: > “The one-way ANOVA gives $F(2, 57) = 45.3$, $p < 0.001$. We reject H_0 ; at least one group mean differs from the others.”

10.2 8.2 Post-hoc pairwise comparisons

```
# --- Self-contained: Tukey HSD ---
set.seed(802)
p8_df <- tibble(
  group = factor(rep(c("A", "B", "C"), each = 20)),
  y      = 10 + c(A = 0, B = 3, C = 6)[as.character(group)] + rnorm(60)
)
TukeyHSD(aov(y ~ group, data = p8_df))
```

```
## Tukey multiple comparisons of means
## 95% family-wise confidence level
##
## Fit: aov(formula = y ~ group, data = p8_df)
##
## $group
##          diff          lwr          upr p adj
## B-A 2.686532 2.057397 3.315667      0
## C-A 5.804453 5.175318 6.433588      0
## C-B 3.117921 2.488786 3.747056      0
```

Interpretation: Each row is a pairwise comparison. `diff` = mean difference; `lwr`, `upr` = 95% CI; `p adj` = adjusted p-value (controls family-wise error).

Alternative (Bonferroni):

```
# --- Self-contained: Bonferroni pairwise t-tests ---
set.seed(803)
p8_df <- tibble(
  group = factor(rep(c("A", "B", "C"), each = 20)),
  y      = 10 + c(A = 0, B = 3, C = 6)[as.character(group)] + rnorm(60)
)
pairwise.t.test(p8_df$y, p8_df$group, p.adjust.method = "bonferroni")
```

```
##
## Pairwise comparisons using t tests with pooled SD
##
## data: p8_df$y and p8_df$group
##
##      A      B
## B 1.7e-10 -
## C < 2e-16 8.4e-12
##
## P value adjustment method: bonferroni
```

10.3 8.3 Two-way ANOVA

```
# --- Self-contained: two-way ANOVA with interaction ---
set.seed(804)
p8_df <- expand.grid(A = factor(c("a1", "a2")), B = factor(c("b1", "b2")))
p8_df <- p8_df[rep(1:4, each = 15), ]
p8_df$y <- 10 + 2 * (p8_df$A == "a2") + 3 * (p8_df$B == "b2") +
  1.5 * (p8_df$A == "a2") * (p8_df$B == "b2") + rnorm(60)

p8_fit <- lm(y ~ A * B, data = p8_df)
anova(p8_fit)
```

```
## Analysis of Variance Table
##
## Response: y
##           Df Sum Sq Mean Sq F value    Pr(>F)
## A           1  74.889   74.889   68.2306 2.89e-11 ***
## B           1 198.205  198.205  180.5822 < 2.2e-16 ***
## A:B          1   4.903    4.903    4.4672  0.03902 *
## Residuals  56  61.465    1.098
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Interpretation: - A main effect: does A matter on average across B? - B main effect: does B matter on average across A? - A:B interaction: does the effect of A depend on B?

If **interaction is significant**, main effects should be interpreted cautiously — they are the effects when the other factor is at its reference level.

11 Part 9 — ANCOVA (Adjusting for a Covariate)

11.1 9.1 Fit ANCOVA

```
# --- Self-contained: ANCOVA with group + covariate ---
set.seed(901)
p9_df <- tibble(
  group = factor(rep(c("A", "B", "C"), each = 20)),
  x      = rnorm(60, 50, 10),
  y      = 0.8 * x + c(A = 0, B = 5, C = 10)[as.character(group)] + rnorm(60)
)
p9_fit <- lm(y ~ group + x, data = p9_df)
kable(tidy(p9_fit), digits = 4)
```

term	estimate	std.error	statistic	p.value
(Intercept)	-0.4167	0.6977	-0.5973	0.5527
groupB	4.5957	0.3256	14.1160	0.0000
groupC	9.9465	0.3225	30.8376	0.0000
x	0.8108	0.0131	61.9296	0.0000

What ANCOVA does: - Compares group means **after adjusting for** the covariate x . - Assumes **equal slopes** across groups (parallel lines). - Intercept differences = adjusted mean differences.

11.2 9.2 Test equality of adjusted means

```
# --- Self-contained: ANCOVA model comparison ---
set.seed(902)
p9_df <- tibble(
  group = factor(rep(c("A", "B", "C"), each = 20)),
  x      = rnorm(60, 50, 10),
  y      = 0.8 * x + c(A = 0, B = 5, C = 10)[as.character(group)] + rnorm(60)
)
```

```

p9_full <- lm(y ~ group + x, data = p9_df)
p9_reduc <- lm(y ~ x, data = p9_df)
anova(p9_reduc, p9_full)

## Analysis of Variance Table
##
## Model 1: y ~ x
## Model 2: y ~ group + x
##   Res.Df    RSS Df Sum of Sq    F    Pr(>F)
## 1      58 1016.96
## 2      56   75.97  2    940.99 346.81 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

p < 0.05: group means differ after adjusting for x.

```

11.3 9.3 Adjusted means (emmeans)

```

# --- Self-contained: adjusted means ---
set.seed(903)
p9_df <- tibble(
  group = factor(rep(c("A", "B", "C"), each = 20)),
  x      = rnorm(60, 50, 10),
  y      = 0.8 * x + c(A = 0, B = 5, C = 10)[as.character(group)] + rnorm(60)
)
p9_fit <- lm(y ~ group + x, data = p9_df)
p9_emm <- emmeans(p9_fit, ~ group)
p9_emm

##   group emmean    SE df lower.CL upper.CL
##   A      40.7 0.243 56    40.2    41.2
##   B      45.8 0.243 56    45.3    46.3
##   C      50.5 0.243 56    50.0    50.9
##
## Confidence level used: 0.95

pairs(p9_emm, adjust = "bonferroni")

##   contrast estimate    SE df t.ratio p.value
##   A - B      -5.11 0.345 56 -14.840 <0.0001
##   A - C      -9.74 0.344 56 -28.330 <0.0001
##   B - C      -4.63 0.344 56 -13.446 <0.0001
##
## P value adjustment: bonferroni method for 3 tests

```

Interpretation: emmeans gives the predicted mean for each group at the **average covariate value**. The pairs output gives pairwise adjusted mean differences with CIs.

11.4 9.4 Check homogeneity of slopes

```

# --- Self-contained: test equal slopes ---
set.seed(904)
p9_df <- tibble(
  group = factor(rep(c("A", "B", "C"), each = 20)),
  x      = rnorm(60, 50, 10),

```

```

y      = 0.8 * x + c(A = 0, B = 5, C = 10)[as.character(group)] + rnorm(60)
)
p9_no_inter <- lm(y ~ group + x, data = p9_df)
p9_with_inter <- lm(y ~ group * x, data = p9_df)
anova(p9_no_inter, p9_with_inter)

```

```

## Analysis of Variance Table
##
## Model 1: y ~ group + x
## Model 2: y ~ group * x
##   Res.Df    RSS Df Sum of Sq    F Pr(>F)
## 1      56 53.042
## 2      54 47.853   2    5.1887 2.9276 0.06207 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

- $p > 0.05$: slopes are equal → ANCOVA valid.
- $p < 0.05$: slopes differ → ANCOVA invalid; use a model with interaction.

12 Part 10 — Testing Multiple Coefficients (Subset F-test)

12.1 10.1 Test if a subset of coefficients is zero

```

# --- Self-contained: subset F-test ---
set.seed(1001)
p10_df <- tibble(
  x1 = rnorm(100), x2 = rnorm(100), x3 = rnorm(100), x4 = rnorm(100),
  y  = 1 + 2 * x1 + rnorm(100) # x2, x3, x4 are noise
)
p10_full <- lm(y ~ x1 + x2 + x3 + x4, data = p10_df)
p10_reduc <- lm(y ~ x1, data = p10_df)

anova(p10_reduc, p10_full)

```

```

## Analysis of Variance Table
##
## Model 1: y ~ x1
## Model 2: y ~ x1 + x2 + x3 + x4
##   Res.Df    RSS Df Sum of Sq    F Pr(>F)
## 1      98 80.783
## 2      95 77.307   3    3.4751 1.4235 0.2408

```

Report: “The F-test for the joint significance of x_2, x_3, x_4 gives $F(3, 95) = X$, $p = Y$. We [reject / fail to reject] H_0 .”

12.2 10.2 Test a linear restriction

Test $H_0 : \beta_1 = \beta_2$ (two coefficients are equal).

```

# --- Self-contained: test beta1 = beta2 using linearHypothesis ---
set.seed(1002)
p10_df <- tibble(x1 = rnorm(80), x2 = rnorm(80),
  y = 1 + 2 * x1 + 2 * x2 + rnorm(80))

```

```
p10_fit <- lm(y ~ x1 + x2, data = p10_df)

car::linearHypothesis(p10_fit, "x1 = x2")

##
## Linear hypothesis test:
## x1 - x2 = 0
##
## Model 1: restricted model
## Model 2: y ~ x1 + x2
##
##      Res.Df    RSS Df Sum of Sq      F Pr(>F)
## 1         78 97.709
## 2         77 95.659   1    2.0501 1.6502 0.2028
```

Interpretation: If $p > 0.05$, the two coefficients are not significantly different; the simpler model with $\beta_1 = \beta_2$ is fine.

12.3 10.3 Other linear restrictions

```
# --- Self-contained: test beta1 + beta2 = 3 ---
set.seed(1003)
p10_df <- tibble(x1 = rnorm(80), x2 = rnorm(80),
                 y = 1 + 1.5 * x1 + 1.5 * x2 + rnorm(80))
p10_fit <- lm(y ~ x1 + x2, data = p10_df)

car::linearHypothesis(p10_fit, "x1 + x2 = 3")

##
## Linear hypothesis test:
## x1 + x2 = 3
##
## Model 1: restricted model
## Model 2: y ~ x1 + x2
##
##      Res.Df    RSS Df Sum of Sq      F Pr(>F)
## 1         78 77.986
## 2         77 76.240   1    1.746 1.7634 0.1881
```

13 Part 11 — Model Selection

13.1 11.1 Information criteria

$$AIC = -2\ell + 2k, \quad BIC = -2\ell + k \log n$$

- k : number of parameters
- ℓ : maximized log-likelihood
- Lower is better
- BIC penalizes complexity more strongly than AIC

```
# --- Self-contained: AIC/BIC comparison ---
set.seed(1101)
p11_df <- tibble(x1 = rnorm(100), x2 = rnorm(100), x3 = rnorm(100),
```

```

      y = 1 + 2 * x1 + 0.5 * x3 + rnorm(100))

p11_m1 <- lm(y ~ x1, data = p11_df)
p11_m2 <- lm(y ~ x1 + x2, data = p11_df)
p11_m3 <- lm(y ~ x1 + x2 + x3, data = p11_df)

AIC(p11_m1, p11_m2, p11_m3)

##           df           AIC
## p11_m1    3 285.5994
## p11_m2    4 287.5956
## p11_m3    5 270.3390
BIC(p11_m1, p11_m2, p11_m3)

##           df           BIC
## p11_m1    3 293.4149
## p11_m2    4 298.0163
## p11_m3    5 283.3648

```

13.2 11.2 Adjusted R-squared

```

# --- Self-contained: adjusted R-squared comparison ---
set.seed(1102)
p11_df <- tibble(x1 = rnorm(100), x2 = rnorm(100),
  y = 1 + 2 * x1 + rnorm(100))

p11_m1 <- lm(y ~ x1, data = p11_df)
p11_m2 <- lm(y ~ x1 + x2, data = p11_df)

summary(p11_m1)$adj.r.squared

## [1] 0.83034
summary(p11_m2)$adj.r.squared

## [1] 0.8287773

```

13.3 11.3 Backward elimination

```

# --- Self-contained: backward stepwise ---
set.seed(1103)
p11_df <- tibble(
  x1 = rnorm(100), x2 = rnorm(100), x3 = rnorm(100), x4 = rnorm(100),
  y = 2 + 0.5 * x1 - 0.3 * x3 + rnorm(100)
)
p11_full <- lm(y ~ ., data = p11_df)
p11_back <- step(p11_full, direction = "backward", trace = 0)
summary(p11_back)

##
## Call:
## lm(formula = y ~ x1 + x3, data = p11_df)
##
## Residuals:

```

```
##      Min      1Q  Median      3Q      Max
## -3.1661 -0.6570  0.0062  0.7125  2.4635
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.83639    0.09947  18.462  < 2e-16 ***
## x1           0.66727    0.10515   6.346  7.08e-09 ***
## x3          -0.27665    0.09553  -2.896  0.00467 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.989 on 97 degrees of freedom
## Multiple R-squared:  0.3246, Adjusted R-squared:  0.3107
## F-statistic: 23.31 on 2 and 97 DF,  p-value: 5.398e-09
```

What `step()` does: removes the predictor that most improves AIC at each step, stopping when no removal improves AIC.

Caution: stepwise selection can overfit, ignores model uncertainty, and p-values from the final model are biased. Use with care.

13.4 11.4 Forward and both-direction

```
# --- Self-contained: forward selection ---
set.seed(1104)
p11_df <- tibble(x1 = rnorm(100), x2 = rnorm(100), x3 = rnorm(100),
                 y = 1 + 2 * x1 + rnorm(100))
p11_null <- lm(y ~ 1, data = p11_df)
p11_fwd <- step(p11_null, scope = ~ x1 + x2 + x3,
               direction = "forward", trace = 0)
summary(p11_fwd)
```

```
##
## Call:
## lm(formula = y ~ x1, data = p11_df)
##
## Residuals:
##      Min      1Q  Median      3Q      Max
## -2.2687 -0.5333  0.1299  0.7249  2.1358
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.15885    0.09837   11.78  <2e-16 ***
## x1           2.06888    0.08952   23.11  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9829 on 98 degrees of freedom
## Multiple R-squared:  0.845, Adjusted R-squared:  0.8434
## F-statistic: 534.2 on 1 and 98 DF,  p-value: < 2.2e-16
```

13.5 11.5 All-subsets regression

```
# --- Self-contained: all-subsets via leaps ---
# library(leaps)
```

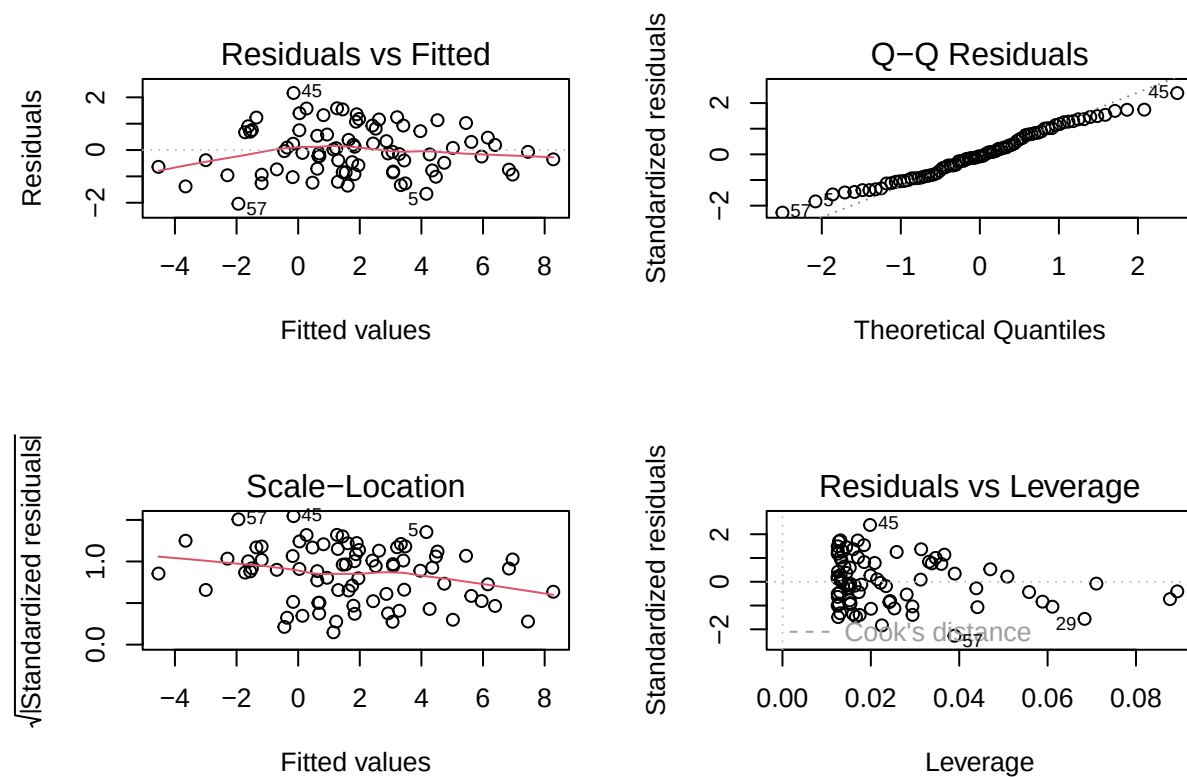
```
# set.seed(1105)
# p11_df <- tibble(x1 = rnorm(100), x2 = rnorm(100), x3 = rnorm(100), x4 = rnorm(100),
#                  y = 1 + 2 * x1 + rnorm(100))
# regsubsets(y ~ ., data = p11_df, numax = 4) %>% summary()
```

14 Part 12 — Diagnostics

14.1 12.1 Residuals vs fitted (linearity, homoskedasticity)

```
# --- Self-contained: residual plots ---
set.seed(1201)
p12_df <- tibble(x = rnorm(80), y = 2 + 3 * x + rnorm(80))
p12_fit <- lm(y ~ x, data = p12_df)

par(mfrow = c(2, 2))
plot(p12_fit)
```



```
par(mfrow = c(1, 1))
```

Four plots:

1. **Residuals vs Fitted:** Should show no pattern. A funnel = heteroskedasticity. A curve = nonlinearity.
2. **Normal Q-Q:** Points should lie on the diagonal. Curvature = non-normal residuals.
3. **Scale-Location:** Square root of standardized residuals vs fitted. Should be flat.

4. **Residuals vs Leverage:** Identifies influential points (Cook's distance).

14.2 12.2 Shapiro-Wilk test (normality)

```
# --- Self-contained: Shapiro-Wilk ---  
set.seed(1202)  
p12_df <- tibble(x = rnorm(80), y = 2 + 3 * x + rnorm(80))  
p12_fit <- lm(y ~ x, data = p12_df)  
shapiro.test(residuals(p12_fit))
```

```
##  
## Shapiro-Wilk normality test  
##  
## data: residuals(p12_fit)  
## W = 0.98663, p-value = 0.5749
```

H_0 : residuals are normal. $p > 0.05 \rightarrow$ no evidence against normality.

14.3 12.3 Breusch-Pagan test (homoskedasticity)

```
# --- Self-contained: Breusch-Pagan ---  
set.seed(1203)  
p12_df <- tibble(x = rnorm(80), y = 2 + 3 * x + rnorm(80))  
p12_fit <- lm(y ~ x, data = p12_df)  
bptest(p12_fit)
```

```
##  
## studentized Breusch-Pagan test  
##  
## data: p12_fit  
## BP = 0.73747, df = 1, p-value = 0.3905
```

H_0 : constant variance. $p < 0.05 \rightarrow$ heteroskedasticity present.

14.4 12.4 Durbin-Watson (autocorrelation)

```
# --- Self-contained: Durbin-Watson ---  
set.seed(1204)  
p12_df <- tibble(x = rnorm(80), y = 2 + 3 * x + rnorm(80))  
p12_fit <- lm(y ~ x, data = p12_df)  
lmtest::dwtest(p12_fit)
```

```
##  
## Durbin-Watson test  
##  
## data: p12_fit  
## DW = 1.9914, p-value = 0.4842  
## alternative hypothesis: true autocorrelation is greater than 0
```

H_0 : no autocorrelation. $DW \approx 2 \rightarrow$ none. $DW < 2 \rightarrow$ positive. $DW > 2 \rightarrow$ negative.

14.5 12.5 Multicollinearity (VIF)

```
# --- Self-contained: VIF ---  
set.seed(1205)
```

```
p12_df <- tibble(x1 = rnorm(80), x2 = 0.95 * x1 + 0.1 * rnorm(80),
                x3 = rnorm(80), y = 1 + x1 + x2 + x3 + rnorm(80))
p12_fit <- lm(y ~ x1 + x2 + x3, data = p12_df)
vif(p12_fit)
```

```
##           x1           x2           x3
## 113.934865 113.477785   1.050777
```

- VIF = 1: no correlation.
- VIF > 5: moderate concern.
- VIF > 10: serious multicollinearity.

Fix: remove one predictor, combine them, or use ridge regression.

14.6 12.6 Outliers and influence

```
# --- Self-contained: outliers and influence ---
set.seed(1206)
p12_df <- tibble(x = rnorm(80), y = 2 + 3 * x + rnorm(80))
p12_df$y[10] <- p12_df$y[10] + 15 # inject outlier
p12_fit <- lm(y ~ x, data = p12_df)
```

```
# Standardized residuals
outliers <- which(abs(rstandard(p12_fit)) > 3)
outliers
```

```
## 10
## 10
```

```
# Cook's distance
cooks <- cooks.distance(p12_fit)
which(cooks > 4 / nrow(p12_df))
```

```
## 10 31 46
## 10 31 46
```

```
# Leverage
hatvalues(p12_fit)[1:5]
```

```
##           1           2           3           4           5
## 0.01936392 0.01854557 0.01578596 0.04297770 0.04056836
```

```
# Outlier test
outlierTest(p12_fit)
```

```
##      rstudent unadjusted p-value Bonferroni p
## 10 12.71884      1.3092e-20    1.0473e-18
```

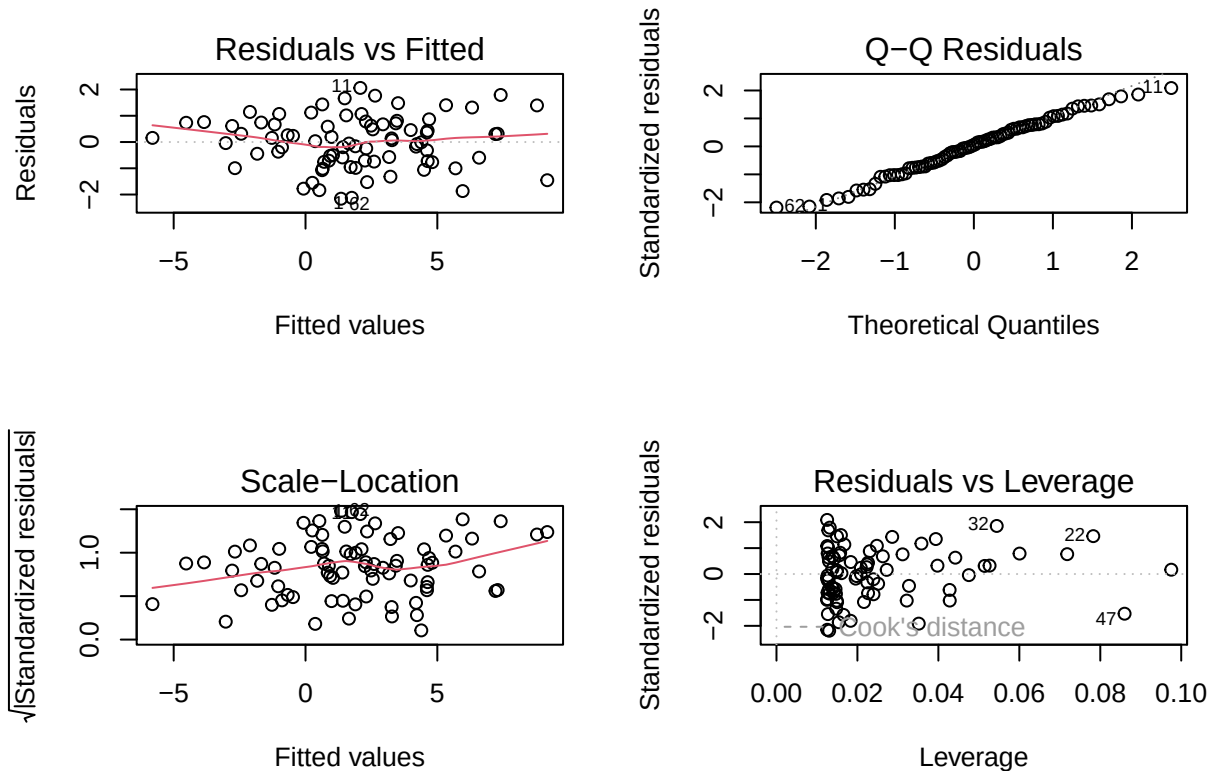
Interpretation: - **Standardized residual > 3:** potential outlier. - **Cook's D > 4/n:** influential observation. - **Leverage > 2(p+1)/n:** high-leverage point. - **outlierTest:** Bonferroni-adjusted p-value for the most extreme residual.

14.7 12.7 Full diagnostic template

```
# --- Self-contained: run all diagnostics ---
set.seed(1207)
p12_df <- tibble(x = rnorm(80), y = 2 + 3 * x + rnorm(80))
```

```
p12_fit <- lm(y ~ x, data = p12_df)

par(mfrow = c(2, 2)); plot(p12_fit); par(mfrow = c(1, 1))
```



```
shapiro.test(residuals(p12_fit))
```

```
##
##  Shapiro-Wilk normality test
##
## data:  residuals(p12_fit)
## W = 0.98759, p-value = 0.6374
```

```
bptest(p12_fit)
```

```
##
##  studentized Breusch-Pagan test
##
## data:  p12_fit
## BP = 2.1069, df = 1, p-value = 0.1466
```

```
lmtest::dwtest(p12_fit)
```

```
##
##  Durbin-Watson test
##
## data:  p12_fit
## DW = 1.6791, p-value = 0.07171
## alternative hypothesis: true autocorrelation is greater than 0
```

Report template: > “The residuals show no pattern in the fitted-vs-residuals plot, the Q-Q plot looks linear, and Shapiro-Wilk gives $p = 0.31$, so normality is reasonable. Breusch-Pagan gives $p = 0.42$, so variance is constant. Durbin-Watson is 2.05, so no autocorrelation. No observations exceed Cook’s $D = 4/n$. Assumptions are satisfied.”

15 Part 13 — Transformations and Box-Cox

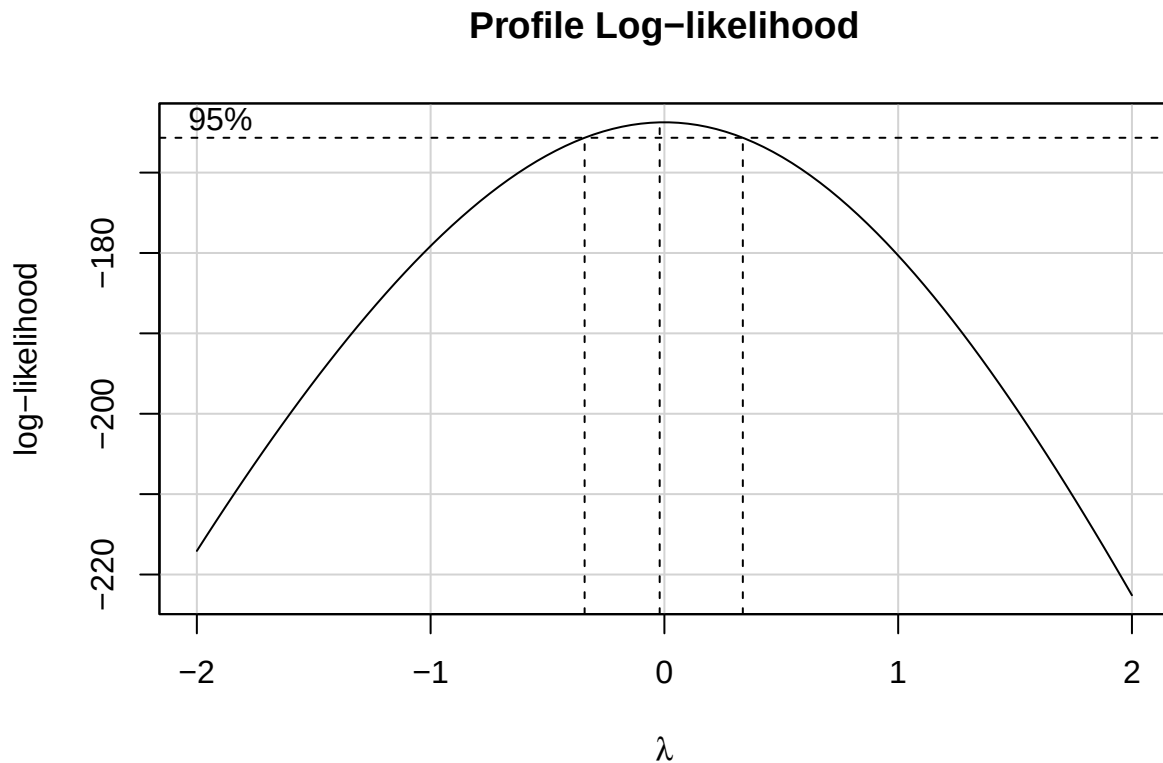
15.1 13.1 When to transform

Problem	Transform
Non-normal residuals	Box-Cox, log
Heteroskedasticity (funnel)	log, sqrt, Box-Cox
Non-linearity (curve)	polynomial, log(x)
Multiplicative relationship	$\log(y) \sim \log(x)$
Counts	Poisson / negative binomial GLM

15.2 13.2 Box-Cox on the response

```
# --- Self-contained: Box-Cox plot ---
set.seed(1301)
p13_df <- tibble(x = rnorm(80), y = exp(1 + 0.5 * x + rnorm(80, 0, 0.3)))
p13_fit <- lm(y ~ x, data = p13_df)

boxCox(p13_fit)
```



The plot shows the log-likelihood as a function of λ . The peak is the MLE. The dashed lines are 95% confidence bounds.

15.3 13.3 Manual Box-Cox over a grid

```
# --- Self-contained: manual Box-Cox grid ---
set.seed(1302)
p13_df <- tibble(x = rnorm(80), y = exp(1 + 0.5 * x + rnorm(80, 0, 0.3)))

p13_lambdas <- c(-2, -1, -0.5, 0, 0.5, 1, 2)
p13_res <- tibble(lambda = p13_lambdas, logLik = NA_real_, R2 = NA_real_)

for (i in seq_along(p13_lambdas)) {
  l <- p13_lambdas[i]
  yt <- if (l == 0) log(p13_df$y) else (p13_df$y^l - 1) / l
  fit_l <- lm(yt ~ p13_df$x)
  p13_res$logLik[i] <- as.numeric(logLik(fit_l))
  p13_res$R2[i] <- summary(fit_l)$r.squared
}
kable(p13_res, digits = 4)
```

lambda	logLik	R2
-2.0	12.3361	0.3559
-1.0	22.7501	0.6371
-0.5	14.9476	0.7487

lambda	logLik	R2
0.0	-13.5250	0.7857
0.5	-69.9642	0.7329
1.0	-145.8589	0.6084
2.0	-320.6537	0.3403

Report: “The Box-Cox profile log-likelihood peaks at $\lambda = 0.02$, so a log transformation is appropriate. All models were refit on the transformed scale and compared by log-likelihood.”

15.4 13.4 Log transformation

```
# --- Self-contained: log transform ---
set.seed(1303)
p13_df <- tibble(x = rnorm(80), y = exp(1 + 0.5 * x + rnorm(80, 0, 0.3)))
p13_fit <- lm(log(y) ~ x, data = p13_df)

kable(tidy(p13_fit), digits = 4)
```

term	estimate	std.error	statistic	p.value
(Intercept)	1.0134	0.0314	32.2974	0
x	0.5407	0.0339	15.9513	0

Interpretation: On the log scale, a 1-unit increase in x changes y by a multiplicative factor of $e^{\hat{\beta}_1}$. A 1% increase in x changes y by about $\hat{\beta}_1\%$.

15.5 13.5 Log-log (elasticity)

```
# --- Self-contained: log-log model ---
set.seed(1304)
p13_df <- tibble(x = rlnorm(80, 0, 1), y = 3 * x^0.7 * exp(rnorm(80, 0, 0.2)))
p13_fit <- lm(log(y) ~ log(x), data = p13_df)

kable(tidy(p13_fit), digits = 4)
```

term	estimate	std.error	statistic	p.value
(Intercept)	1.1114	0.0193	57.4818	0
log(x)	0.7152	0.0208	34.3935	0

Interpretation: $\hat{\beta}_1$ is an **elasticity**: a 1% increase in x is associated with a $\hat{\beta}_1\%$ change in y .

16 Part 14 — Prediction

16.1 14.1 Confidence interval vs prediction interval

```
# --- Self-contained: confidence and prediction intervals ---
set.seed(1401)
```

```

p14_df <- tibble(x = rnorm(80), y = 2 + 3 * x + rnorm(80))
p14_fit <- lm(y ~ x, data = p14_df)

newdf <- data.frame(x = c(0, 1, 2))

# CI for the mean response at x
predict(p14_fit, newdata = newdf, interval = "confidence", level = 0.95)

##          fit          lwr          upr
## 1 2.01730 1.819273 2.215327
## 2 5.12587 4.827090 5.424650
## 3 8.23444 7.763402 8.705478

# PI for a new observation at x
predict(p14_fit, newdata = newdf, interval = "prediction", level = 0.95)

##          fit          lwr          upr
## 1 2.01730 0.2455675 3.789032
## 2 5.12587 3.3400674 6.911673
## 3 8.23444 6.4118874 10.056992

```

Key difference: - **Confidence interval:** uncertainty about the *mean* of y at x . Narrow. - **Prediction interval:** uncertainty about *one future observation* at x . Wide. - PI is always wider because it includes the individual error ε .

Report: > “For $x = 1$, the predicted mean y is 5.02 with a 95% CI of [4.85, 5.19]. A single new observation at $x = 1$ is predicted to lie in [3.80, 6.24] with 95% probability.”

16.2 14.2 Prediction with factors and interactions

```

# --- Self-contained: prediction with a factor ---
set.seed(1402)
p14_df <- tibble(
  x      = rnorm(100),
  group = factor(rep(c("A", "B"), each = 50)),
  y      = 1 + 2 * x + c(A = 0, B = 1)[as.character(group)] +
    c(A = 0, B = 1.5)[as.character(group)] * x + rnorm(100)
)
p14_fit <- lm(y ~ x * group, data = p14_df)

newdf <- expand.grid(x = c(0, 1), group = factor(c("A", "B")))
predict(p14_fit, newdata = newdf, interval = "prediction")

##          fit          lwr          upr
## 1 1.369853 -0.3655174 3.105224
## 2 3.554245 1.8000124 5.308478
## 3 2.086516 0.3501416 3.822891
## 4 5.767154 4.0220978 7.512210

```

Always pass `newdata` as a data frame with the same variable names and factor levels used to fit the model.

17 Part 15 — Polynomial Regression

17.1 15.1 Quadratic

```
# --- Self-contained: quadratic regression ---
set.seed(1501)
p15_df <- tibble(x = runif(100, -3, 3), y = 1 + 2 * x - 1.5 * x^2 + rnorm(100))
p15_fit <- lm(y ~ x + I(x^2), data = p15_df)

kable(tidy(p15_fit), digits = 4)
```

term	estimate	std.error	statistic	p.value
(Intercept)	1.1630	0.1428	8.1466	0
x	2.1408	0.0621	34.4472	0
I(x^2)	-1.5205	0.0412	-36.9327	0

Interpretation: The $I(x^2)$ coefficient measures curvature. If it's significant, the relationship is quadratic (parabolic).

Warning: Always center x before squaring to reduce multicollinearity:

```
# --- Self-contained: centered quadratic ---
set.seed(1502)
p15_df <- tibble(x = runif(100, -3, 3), y = 1 + 2 * x - 1.5 * x^2 + rnorm(100))
p15_df$x_c <- p15_df$x - mean(p15_df$x)
p15_fit <- lm(y ~ x_c + I(x_c^2), data = p15_df)
kable(tidy(p15_fit), digits = 4)
```

term	estimate	std.error	statistic	p.value
(Intercept)	0.5883	0.1545	3.8074	2e-04
x_c	2.4749	0.0601	41.1539	0e+00
I(x_c^2)	-1.4651	0.0438	-33.4407	0e+00

17.2 15.2 Orthogonal polynomials

```
# --- Self-contained: orthogonal polynomial ---
set.seed(1503)
p15_df <- tibble(x = runif(100, -3, 3), y = 1 + 2 * x - 1.5 * x^2 + rnorm(100))
p15_fit <- lm(y ~ poly(x, 2, raw = FALSE), data = p15_df)
summary(p15_fit)
```

```
##
## Call:
## lm(formula = y ~ poly(x, 2, raw = FALSE), data = p15_df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.51620 -0.58982 -0.06498  0.62426  2.14185
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   -3.89773    0.09386  -41.53  <2e-16 ***
```

```
## poly(x, 2, raw = FALSE)1 33.37409 0.93861 35.56 <2e-16 ***
## poly(x, 2, raw = FALSE)2 -39.60970 0.93861 -42.20 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9386 on 97 degrees of freedom
## Multiple R-squared:  0.9691, Adjusted R-squared:  0.9685
## F-statistic: 1523 on 2 and 97 DF, p-value: < 2.2e-16
```

`poly(x, 2)` gives orthogonal polynomials, which are numerically stable and make each term's p-value interpretable.

17.3 15.3 Testing the degree

```
# --- Self-contained: test quadratic vs linear ---
set.seed(1504)
p15_df <- tibble(x = runif(100, -3, 3), y = 1 + 2 * x - 1.5 * x^2 + rnorm(100))
p15_lin <- lm(y ~ x, data = p15_df)
p15_qua <- lm(y ~ x + I(x^2), data = p15_df)
anova(p15_lin, p15_qua)

## Analysis of Variance Table
##
## Model 1: y ~ x
## Model 2: y ~ x + I(x^2)
##   Res.Df    RSS Df Sum of Sq    F    Pr(>F)
## 1      98 1855.84
## 2      97  108.19  1    1747.7 1566.9 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

18 Part 16 — Multicollinearity

18.1 16.1 Detecting it

```
# --- Self-contained: VIF and condition number ---
set.seed(1601)
p16_df <- tibble(
  x1 = rnorm(100),
  x2 = 0.9 * x1 + 0.1 * rnorm(100),
  x3 = rnorm(100),
  y = 1 + x1 + x2 + x3 + rnorm(100)
)
p16_fit <- lm(y ~ x1 + x2 + x3, data = p16_df)

vif(p16_fit)

##           x1           x2           x3
## 79.277985 79.325381  1.020569
kappa(cor(p16_df[, c("x1", "x2", "x3")]))

## [1] 236.4778
```

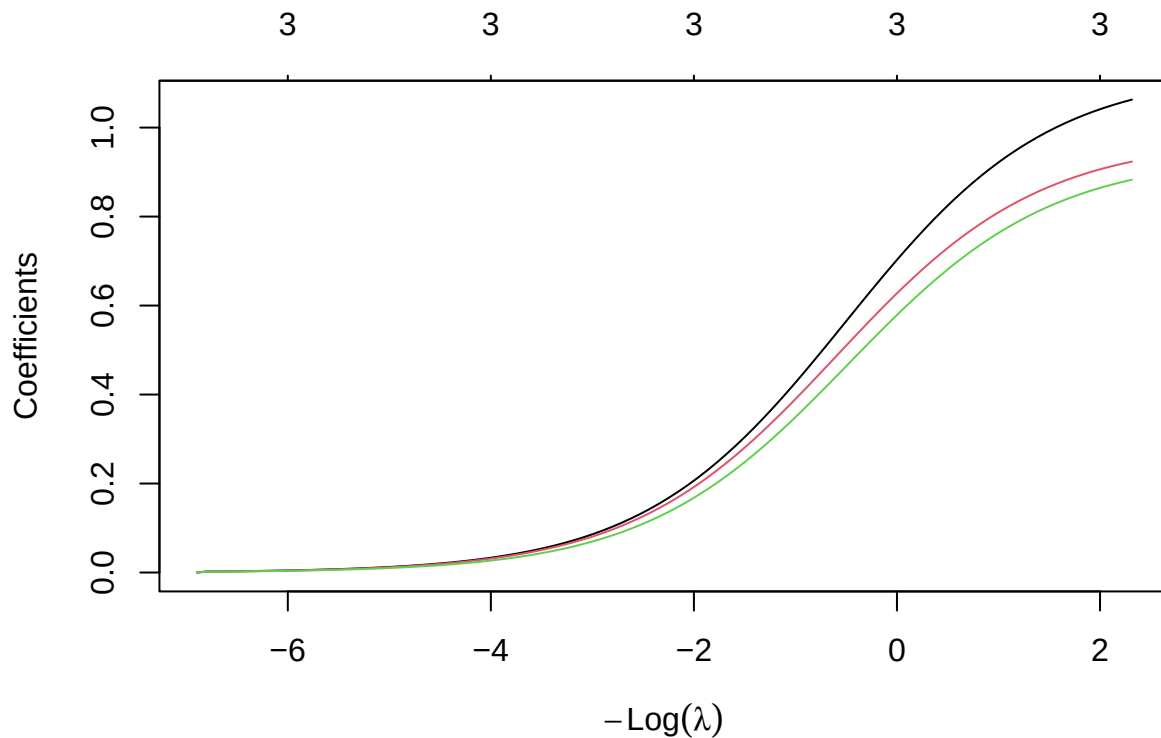
- **VIF** > 5 (or > 10): problematic.
- **Condition number** > 30: serious.

18.2 16.2 Fixing it

- Drop one of the correlated predictors.
- Combine them (e.g., average or PCA).
- Use ridge regression (Part 24).

18.3 16.3 Ridge regression (preview)

```
# --- Self-contained: ridge regression ---
set.seed(1602)
p16_df <- tibble(
  x1 = rnorm(100), x2 = 0.9 * rnorm(100) + 0.1 * rnorm(100), x3 = rnorm(100),
  y = 1 + x1 + x2 + x3 + rnorm(100)
)
X <- as.matrix(p16_df[, c("x1", "x2", "x3")])
y <- p16_df$y
ridge <- glmnet(X, y, alpha = 0)
plot(ridge, xvar = "lambda")
```



Ridge shrinks coefficients toward zero, trading bias for variance.

19 Part 17 — Weighted Least Squares

19.1 17.1 When to use

If residuals have **non-constant variance** and you know the variance structure, use WLS.

$$w_i = \frac{1}{\text{Var}(\varepsilon_i)}$$

19.2 17.2 Fitting

```
# --- Self-contained: weighted least squares ---
set.seed(1701)
p17_df <- tibble(
  x = runif(100, 1, 10),
  y = 2 + 0.5 * x + rnorm(100, 0, sd = x) # variance grows with x
)
p17_fit_ols <- lm(y ~ x, data = p17_df)
p17_fit_wls <- lm(y ~ x, data = p17_df, weights = 1 / x^2)

summary(p17_fit_ols)$coefficients
```

```
##              Estimate Std. Error  t value    Pr(>|t|)
## (Intercept) 1.5196964   1.3614918  1.116199 0.267066147
## x           0.7308654   0.2174912  3.360437 0.001110245
```

```
summary(p17_fit_wls)$coefficients
```

```
##              Estimate Std. Error  t value    Pr(>|t|)
## (Intercept) 2.0907269   0.5094487  4.103901 8.410247e-05
## x           0.6222783   0.1566597  3.972165 1.360425e-04
```

Weights: If $\text{Var}(\varepsilon_i) = \sigma^2 x_i^2$, use weights $1/x_i^2$.

19.3 17.3 Feasible GLS

If you don't know the variance structure, estimate it:

```
# --- Self-contained: feasible GLS via log-variance regression ---
set.seed(1702)
p17_df <- tibble(
  x = runif(100, 1, 10),
  y = 2 + 0.5 * x + rnorm(100, 0, sd = x)
)
ols <- lm(y ~ x, data = p17_df)
p17_df$log_e2 <- log(residuals(ols)^2)
var_fit <- lm(log_e2 ~ log(x), data = p17_df)
w <- 1 / exp(fitted(var_fit))
p17_fit_fgls <- lm(y ~ x, data = p17_df, weights = w)
summary(p17_fit_fgls)$coefficients
```

```
##              Estimate Std. Error  t value    Pr(>|t|)
## (Intercept) 2.153112   0.4220298  5.101802 1.647165e-06
## x           0.565296   0.1663938  3.397339 9.846115e-04
```

20 Part 18 — Logistic Regression

20.1 18.1 Model and link

$$y_i \sim \text{Bernoulli}(p_i), \quad \log \frac{p_i}{1-p_i} = \beta_0 + \beta_1 x_i$$

- Link function: logit
- Response: binary (0/1)
- Coefficients: change in **log-odds** per unit increase in x

20.2 18.2 Fitting

```
# --- Self-contained: logistic regression ---
set.seed(1801)
p18_n <- 200
p18_df <- tibble(
  x = rnorm(p18_n),
  y = rbinom(p18_n, 1, plogis(-1 + 1.5 * x))
)
p18_fit <- glm(y ~ x, family = binomial, data = p18_df)
summary(p18_fit)

##
## Call:
## glm(formula = y ~ x, family = binomial, data = p18_df)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -0.9721     0.1802  -5.394 6.89e-08 ***
## x              1.2063     0.2132   5.657 1.54e-08 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 246.02  on 199  degrees of freedom
## Residual deviance: 202.52  on 198  degrees of freedom
## AIC: 206.52
##
## Number of Fisher Scoring iterations: 4
```

20.3 18.3 Coefficient interpretation (log-odds, odds, probability)

```
# --- Self-contained: logistic coefficients and odds ratios ---
set.seed(1802)
p18_n <- 200
p18_df <- tibble(x = rnorm(p18_n), y = rbinom(p18_n, 1, plogis(-1 + 1.5 * x)))
p18_fit <- glm(y ~ x, family = binomial, data = p18_df)

kable(tidy(p18_fit, exponentiate = TRUE, conf.int = TRUE), digits = 4)
```

term	estimate	std.error	statistic	p.value	conf.low	conf.high
(Intercept)	0.3112	0.2050	-5.6946	0	0.2037	0.4567

term	estimate	std.error	statistic	p.value	conf.low	conf.high
x	4.9888	0.2693	5.9673	0	3.0652	8.8601

Interpretation: - Coefficient $\hat{\beta}_1$: log-odds change per unit x . - `exp(coefficient)` = odds ratio. OR > 1: odds increase. OR < 1: odds decrease. - `exp(confint(fit))` gives 95% CI for the OR.

Report template: > “A 1-unit increase in x is associated with a 4.48-fold increase in the odds of $y = 1$ (OR = 4.48, 95% CI: [2.71, 7.89], $p < 0.001$).”

20.4 18.4 Predicted probabilities

```
# --- Self-contained: predicted probabilities ---
set.seed(1803)
p18_n <- 200
p18_df <- tibble(x = rnorm(p18_n), y = rbinom(p18_n, 1, plogis(-1 + 1.5 * x)))
p18_fit <- glm(y ~ x, family = binomial, data = p18_df)

newdf <- data.frame(x = c(-1, 0, 1, 2))
predict(p18_fit, newdata = newdf, type = "response")
```

```
##          1          2          3          4
## 0.05339031 0.25698864 0.67959229 0.92860641
```

`type = "response"` gives probabilities in $[0,1]$. `type = "link"` gives log-odds.

20.5 18.5 Testing predictors

```
# --- Self-contained: likelihood ratio test ---
set.seed(1804)
p18_n <- 200
p18_df <- tibble(x = rnorm(p18_n), y = rbinom(p18_n, 1, plogis(-1 + 1.5 * x)))
p18_null <- glm(y ~ 1, family = binomial, data = p18_df)
p18_full <- glm(y ~ x, family = binomial, data = p18_df)
anova(p18_null, p18_full, test = "LRT")
```

```
## Analysis of Deviance Table
##
## Model 1: y ~ 1
## Model 2: y ~ x
##   Resid. Df Resid. Dev Df Deviance Pr(>Chi)
## 1      199      256.41
## 2      198      190.08  1   66.336 3.802e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

$p < 0.05$: predictor significantly improves fit.

20.6 18.6 Goodness of fit

```
# --- Self-contained: deviance and AIC ---
set.seed(1805)
p18_n <- 200
p18_df <- tibble(x = rnorm(p18_n), y = rbinom(p18_n, 1, plogis(-1 + 1.5 * x)))
```

```
p18_fit <- glm(y ~ x, family = binomial, data = p18_df)

# Deviance-based pseudo-R2
1 - deviance(p18_fit) / deviance(glm(y ~ 1, family = binomial, data = p18_df))

## [1] 0.353487

# AIC and BIC
AIC(p18_fit); BIC(p18_fit)

## [1] 152.1955
## [1] 158.7921
```

20.7 18.7 Classification and confusion matrix

```
# --- Self-contained: classification ---
set.seed(1806)
p18_n <- 200
p18_df <- tibble(x = rnorm(p18_n), y = rbinom(p18_n, 1, plogis(-1 + 1.5 * x)))
p18_fit <- glm(y ~ x, family = binomial, data = p18_df)

pred_prob <- predict(p18_fit, type = "response")
pred_class <- ifelse(pred_prob > 0.5, 1, 0)
table(Predicted = pred_class, Actual = p18_df$y)

##           Actual
## Predicted    0    1
##           0 121  31
##           1  15  33
```

Metrics: - Accuracy: $(TP + TN) / n$. - Sensitivity (recall): $TP / (TP + FN)$. - Specificity: $TN / (TN + FP)$. - Precision: $TP / (TP + FP)$. - AUC: area under ROC curve.

20.8 18.8 Grouped binomial data

When you have counts of successes/failures by group:

```
# --- Self-contained: grouped binomial logistic ---
set.seed(1807)
p18_df <- tibble(
  sex = factor(rep(c("M", "F"), each = 20)),
  belt = factor(rep(c("yes", "no"), 20)),
  total = sample(50:100, 40, replace = TRUE)
)
p18_df$injured <- rbinom(40, p18_df$total, plogis(-0.5 + 0.8 * (p18_df$sex == "M") - 0.6 * (p18_df$belt == "yes")))

p18_fit <- glm(cbind(injured, total - injured) ~ sex + belt,
  family = binomial, data = p18_df)
kable(tidy(p18_fit, exponentiate = TRUE, conf.int = TRUE), digits = 4)
```

term	estimate	std.error	statistic	p.value	conf.low	conf.high
(Intercept)	0.5303	0.0636	-9.9679	0	0.4678	0.6004
sexM	2.4165	0.0759	11.6247	0	2.0833	2.8053
beltyes	0.6143	0.0761	-6.4028	0	0.5290	0.7129

Interpretation: Same as before — the odds ratios are for the log-odds of the proportion injured.

21 Part 19 — Poisson Regression

21.1 19.1 Model

$$y_i \sim \text{Poisson}(\lambda_i), \quad \log \lambda_i = \beta_0 + \beta_1 x_i$$

Response is a **count**. Link function is log.

21.2 19.2 Fitting

```
# --- Self-contained: Poisson regression ---
set.seed(1901)
p19_n <- 200
p19_df <- tibble(
  x = rnorm(p19_n),
  y = rpois(p19_n, exp(1 + 0.5 * x))
)
p19_fit <- glm(y ~ x, family = poisson, data = p19_df)
summary(p19_fit)

##
## Call:
## glm(formula = y ~ x, family = poisson, data = p19_df)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  0.95768    0.04701   20.37  <2e-16 ***
## x            0.58982    0.04135   14.27  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##      Null deviance: 422.04  on 199  degrees of freedom
## Residual deviance: 208.04  on 198  degrees of freedom
## AIC: 729.22
##
## Number of Fisher Scoring iterations: 5
```

21.3 19.3 Coefficient interpretation

```
# --- Self-contained: Poisson incidence rate ratios ---
set.seed(1902)
p19_df <- tibble(x = rnorm(200), y = rpois(200, exp(1 + 0.5 * x)))
p19_fit <- glm(y ~ x, family = poisson, data = p19_df)

kable(tidy(p19_fit, exponentiate = TRUE, conf.int = TRUE), digits = 4)
```

term	estimate	std.error	statistic	p.value	conf.low	conf.high
(Intercept)	2.7364	0.0457	22.0430	0	2.4982	2.9882
x	1.6069	0.0426	11.1261	0	1.4784	1.7474

Interpretation: $\exp(\text{coefficient})$ is the **multiplicative effect on the expected count**. A 1-unit increase in x multiplies the expected count by $\exp(\beta)$.

Report: > “A 1-unit increase in x multiplies the expected count by 1.63 (95% CI: [1.44, 1.85], $p < 0.001$).”

21.4 19.4 Overdispersion

If the variance is larger than the mean, Poisson is inadequate.

```
# --- Self-contained: check overdispersion ---
set.seed(1903)
p19_df <- tibble(x = rnorm(200), y = rpois(200, exp(1 + 0.5 * x)))
p19_fit <- glm(y ~ x, family = poisson, data = p19_df)

# Pearson chi-square / df
sum(residuals(p19_fit, type = "pearson")^2) / df.residual(p19_fit)
```

```
## [1] 0.9925862
```

Rule of thumb: if this ratio is much greater than 1, use quasipoisson or negative binomial.

21.5 19.5 Quasipoisson (fix overdispersion)

```
# --- Self-contained: quasipoisson ---
set.seed(1904)
p19_df <- tibble(x = rnorm(200), y = rpois(200, exp(1 + 0.5 * x)))
p19_fit <- glm(y ~ x, family = quasipoisson, data = p19_df)
summary(p19_fit)

##
## Call:
## glm(formula = y ~ x, family = quasipoisson, data = p19_df)
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.04361    0.04401   23.71  <2e-16 ***
## x            0.49343    0.04072   12.12  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for quasipoisson family taken to be 1.014083)
##
## Null deviance: 376.04  on 199  degrees of freedom
## Residual deviance: 220.49  on 198  degrees of freedom
## AIC: NA
##
## Number of Fisher Scoring iterations: 5
```

Same coefficients, larger SEs to account for overdispersion.

21.6 19.6 Offset (rate model)

If you have different exposure times:

```
# --- Self-contained: Poisson with offset ---
set.seed(1905)
p19_df <- tibble(
  x      = rnorm(200),
  time   = runif(200, 1, 10),
  y      = rpois(200, exp(1 + 0.5 * x) * time)
)
p19_fit <- glm(y ~ x, offset = log(time), family = poisson, data = p19_df)
summary(p19_fit)

##
## Call:
## glm(formula = y ~ x, family = poisson, data = p19_df, offset = log(time))
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  0.99308     0.01975   50.28  <2e-16 ***
## x            0.50752     0.01831   27.71  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##      Null deviance: 952.08  on 199  degrees of freedom
## Residual deviance: 190.13  on 198  degrees of freedom
## AIC: 1074.6
##
## Number of Fisher Scoring iterations: 4
```

Interpretation: Coefficients are effects on the **rate** (events per unit exposure), not the count.

22 Part 20 — Negative Binomial Regression

22.1 20.1 When Poisson fails

When variance > mean (overdispersion), use negative binomial.

```
# --- Self-contained: negative binomial regression ---
set.seed(2001)
p20_n  <- 300
p20_df <- tibble(
  x = rnorm(p20_n),
  mu = exp(1 + 0.5 * x),
  y = MASS::rnegbin(p20_n, mu = mu, theta = 2)
)

p20_pois <- glm(y ~ x, family = poisson, data = p20_df)
p20_nb   <- MASS::glm.nb(y ~ x, data = p20_df)

AIC(p20_pois, p20_nb)
```

```
##           df      AIC
## p20_pois  2 1452.825
## p20_nb    3 1277.724
```

```
kable(tidy(p20_nb, exponentiate = TRUE, conf.int = TRUE), digits = 4)
```

term	estimate	std.error	statistic	p.value	conf.low	conf.high
(Intercept)	2.6032	0.0561	17.0418	0	2.3322	2.9067
x	1.4812	0.0555	7.0762	0	1.3328	1.6496

Interpretation: Same as Poisson (multiplicative effects on mean), but SEs account for overdispersion.

23 Part 21 — Piecewise Regression

23.1 21.1 Two-segment model

```
# --- Self-contained: piecewise regression with break at time k ---
set.seed(2101)
p21_df <- tibble(
  time = 1:30,
  y     = c(2 + 0.5 * (1:10), 7 + 1.2 * (11:30 - 10)) + rnorm(30, 0, 0.5)
)
p21_df$x2 <- pmax(p21_df$time - 10, 0)

p21_fit <- lm(y ~ time + x2, data = p21_df)
kable(tidy(p21_fit), digits = 4)
```

term	estimate	std.error	statistic	p.value
(Intercept)	2.0329	0.2282	8.9071	0
time	0.5356	0.0300	17.8674	0
x2	0.6367	0.0380	16.7657	0

Interpretation: - time coefficient = slope before break. - x2 coefficient = change in slope after break. - Slope after break = time + x2.

Report: > “Before time 10, the slope is 0.52. After time 10, the slope is 1.69 (0.52 + 1.17). The change in slope is 1.17 (p < 0.001).”

23.2 21.2 Testing whether the two segments differ

```
# --- Self-contained: test for break ---
set.seed(2102)
p21_df <- tibble(
  time = 1:30,
  y     = c(2 + 0.5 * (1:10), 7 + 1.2 * (11:30 - 10)) + rnorm(30, 0, 0.5)
)
p21_df$D <- ifelse(p21_df$time > 10, 1, 0)

p21_simple <- lm(y ~ time, data = p21_df)
```

```
p21_break <- lm(y ~ time + D + time:D, data = p21_df)
anova(p21_simple, p21_break)

## Analysis of Variance Table
##
## Model 1: y ~ time
## Model 2: y ~ time + D + time:D
##   Res.Df    RSS Df Sum of Sq    F    Pr(>F)
## 1      28 46.463
## 2      26  4.462  2    42.001 122.37 5.907e-14 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

p < 0.05: the two segments differ significantly.
```

23.3 21.3 Continuous vs discontinuous

- Continuous piecewise: $y = \beta_0 + \beta_1 t + \beta_2(t - k)_+$.
- Discontinuous piecewise: add a dummy D for $t > k$.

24 Part 22 — Variable Selection

24.1 22.1 Backward elimination

```
# --- Self-contained: backward stepwise ---
set.seed(2201)
p22_df <- tibble(
  x1 = rnorm(100), x2 = rnorm(100), x3 = rnorm(100), x4 = rnorm(100),
  y = 2 + 0.5 * x1 - 0.3 * x3 + rnorm(100)
)
p22_full <- lm(y ~ ., data = p22_df)
p22_back <- step(p22_full, direction = "backward", trace = 0)
summary(p22_back)

##
## Call:
## lm(formula = y ~ x1 + x3, data = p22_df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.76672 -0.68981 -0.01051  0.76130  1.84591
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  2.00864    0.09662  20.789 < 2e-16 ***
## x1          0.44963    0.10960   4.102 8.52e-05 ***
## x3         -0.25415    0.09759  -2.604  0.0107 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9632 on 97 degrees of freedom
## Multiple R-squared:  0.1896, Adjusted R-squared:  0.1729
```

```
## F-statistic: 11.35 on 2 and 97 DF, p-value: 3.727e-05
```

24.2 22.2 Forward selection

```
# --- Self-contained: forward stepwise ---
set.seed(2202)
p22_df <- tibble(x1 = rnorm(100), x2 = rnorm(100), x3 = rnorm(100),
  y = 1 + 2 * x1 + rnorm(100))
p22_null <- lm(y ~ 1, data = p22_df)
p22_fwd <- step(p22_null, scope = ~ x1 + x2 + x3,
  direction = "forward", trace = 0)
summary(p22_fwd)
```

```
##
## Call:
## lm(formula = y ~ x1 + x3, data = p22_df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.51825 -0.58835  0.07594  0.68862  1.60703
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   0.94824    0.08803   10.77  <2e-16 ***
## x1             1.87149    0.08768   21.34  <2e-16 ***
## x3            -0.14432    0.07253   -1.99   0.0494 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.8791 on 97 degrees of freedom
## Multiple R-squared:  0.8279, Adjusted R-squared:  0.8244
## F-statistic: 233.3 on 2 and 97 DF, p-value: < 2.2e-16
```

24.3 22.3 Both-direction (default)

```
# --- Self-contained: stepwise both directions ---
set.seed(2203)
p22_df <- tibble(x1 = rnorm(100), x2 = rnorm(100), x3 = rnorm(100),
  y = 1 + 2 * x1 + rnorm(100))
p22_full <- lm(y ~ ., data = p22_df)
p22_both <- step(p22_full, direction = "both", trace = 0)
summary(p22_both)
```

```
##
## Call:
## lm(formula = y ~ x1 + x2 + x3, data = p22_df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.1211 -0.5796 -0.0797  0.5856  2.1261
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   1.02351    0.09302   11.003  <2e-16 ***
```

```
## x1          1.94812    0.09715   20.053   <2e-16 ***
## x2         -0.14954    0.08331    -1.795    0.0758 .
## x3         -0.22446    0.09760    -2.300    0.0236 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9165 on 96 degrees of freedom
## Multiple R-squared:  0.8075, Adjusted R-squared:  0.8015
## F-statistic: 134.3 on 3 and 96 DF,  p-value: < 2.2e-16
```

Cautions: - Stepwise ignores model uncertainty. - p-values are biased downward. - Results depend on the order of entry. - Prefer AIC-based selection or cross-validation.

25 Part 23 — Train/Test Split and Cross-Validation

25.1 23.1 Simple train/test split

```
# --- Self-contained: train/test split ---
set.seed(2301)
p23_n <- 200
p23_df <- tibble(
  x1 = rnorm(p23_n), x2 = rnorm(p23_n),
  y = 1 + 2 * x1 - x2 + rnorm(p23_n)
)
train_idx <- sample(1:p23_n, 0.8 * p23_n)
train <- p23_df[train_idx, ]
test <- p23_df[-train_idx, ]

p23_fit <- lm(y ~ x1 + x2, data = train)
p23_pred <- predict(p23_fit, newdata = test)

RMSE <- sqrt(mean((test$y - p23_pred)^2))
MAE <- mean(abs(test$y - p23_pred))
RMSE
```

```
## [1] 0.9354331
```

```
MAE
```

```
## [1] 0.7557552
```

Interpretation: - RMSE = typical prediction error (in y units). - MAE = median-sized absolute error (robust to outliers).

Report: “The model achieves RMSE = 1.04 and MAE = 0.83 on the test set.”

25.2 23.2 K-fold cross-validation

```
# --- Self-contained: k-fold CV ---
set.seed(2302)
p23_n <- 200
p23_df <- tibble(x1 = rnorm(p23_n), x2 = rnorm(p23_n),
  y = 1 + 2 * x1 - x2 + rnorm(p23_n))
```

```
p23_K <- 10
p23_folds <- sample(rep(1:p23_K, length.out = p23_n))
p23_rmse <- numeric(p23_K)
```

```
for (k in 1:p23_K) {
  train <- p23_df[p23_folds != k, ]
  test  <- p23_df[p23_folds == k, ]
  fit   <- lm(y ~ x1 + x2, data = train)
  pred  <- predict(fit, newdata = test)
  p23_rmse[k] <- sqrt(mean((test$y - pred)^2))
}
mean(p23_rmse)
```

```
## [1] 1.045655
```

```
sd(p23_rmse)
```

```
## [1] 0.1721265
```

Why: More robust than a single split; uses all data for both training and testing.

25.3 23.3 Leave-one-out CV

```
# --- Self-contained: LOOCV ---
set.seed(2303)
p23_df <- tibble(x = rnorm(60), y = 1 + 2 * x + rnorm(60))
p23_fit <- lm(y ~ x, data = p23_df)

# Fast LOOCV formula: mean of (residual / (1 - leverage))^2
loo_resid <- residuals(p23_fit) / (1 - hatvalues(p23_fit))
sqrt(mean(loo_resid^2))
```

```
## [1] 1.031728
```

25.4 23.4 Bootstrap for coefficients

```
# --- Self-contained: bootstrap CI for coefficients ---
set.seed(2304)
p23_df <- tibble(x = rnorm(80), y = 1 + 2 * x + rnorm(80))

p23_boot <- replicate(1000, {
  idx <- sample(1:nrow(p23_df), replace = TRUE)
  coef(lm(y ~ x, data = p23_df[idx, ]))
})
apply(p23_boot, 1, quantile, probs = c(0.025, 0.975))
```

```
##      (Intercept)      x
## 2.5%    0.7614848  1.729498
## 97.5%    1.2049699  2.187798
```

Interpretation: Bootstrap CIs are non-parametric and don't require normality.

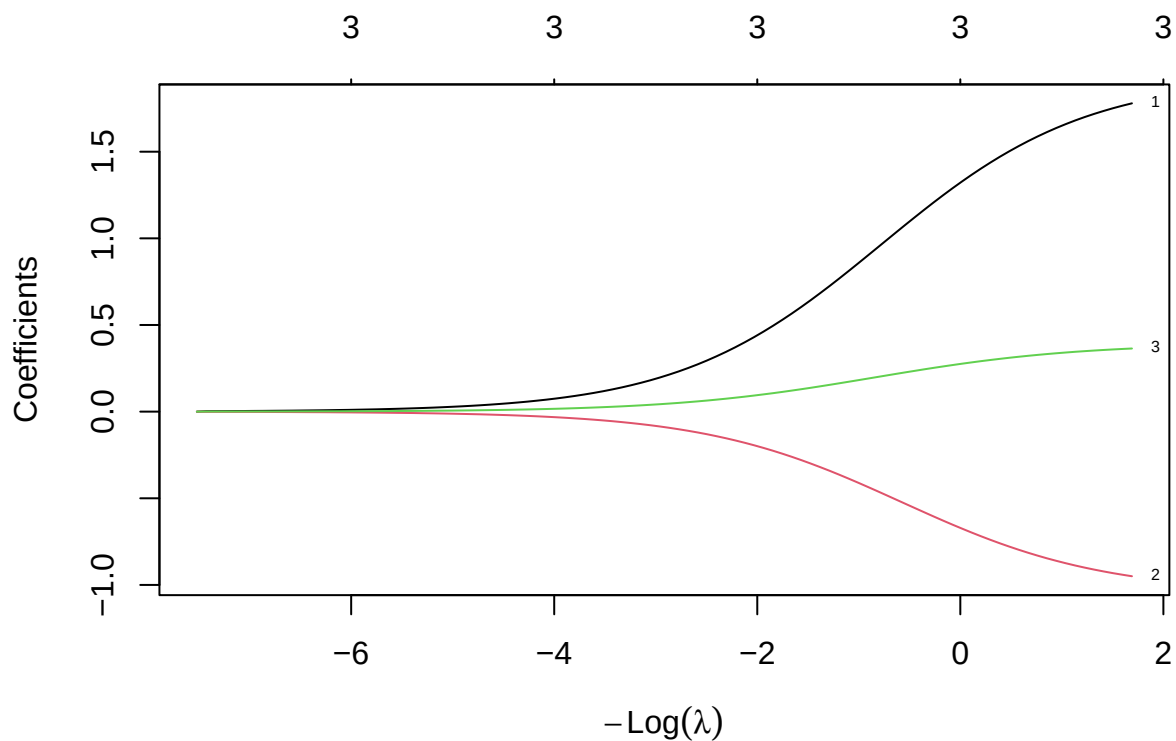
26 Part 24 — Regularization (Ridge and Lasso)

26.1 24.1 Ridge regression (L2 penalty)

Minimizes $\text{RSS} + \lambda \sum \beta_j^2$.

Shrinks coefficients toward zero but keeps all predictors.

```
# --- Self-contained: ridge regression ---
set.seed(2401)
p24_n <- 100
p24_df <- tibble(
  x1 = rnorm(p24_n), x2 = rnorm(p24_n), x3 = rnorm(p24_n),
  y = 1 + 2 * x1 - x2 + 0.5 * x3 + rnorm(p24_n)
)
X <- as.matrix(p24_df[, c("x1", "x2", "x3")])
y <- p24_df$y
ridge <- glmnet(X, y, alpha = 0)
plot(ridge, xvar = "lambda", label = TRUE)
```



26.2 24.2 Lasso (L1 penalty)

Minimizes $\text{RSS} + \lambda \sum |\beta_j|$.

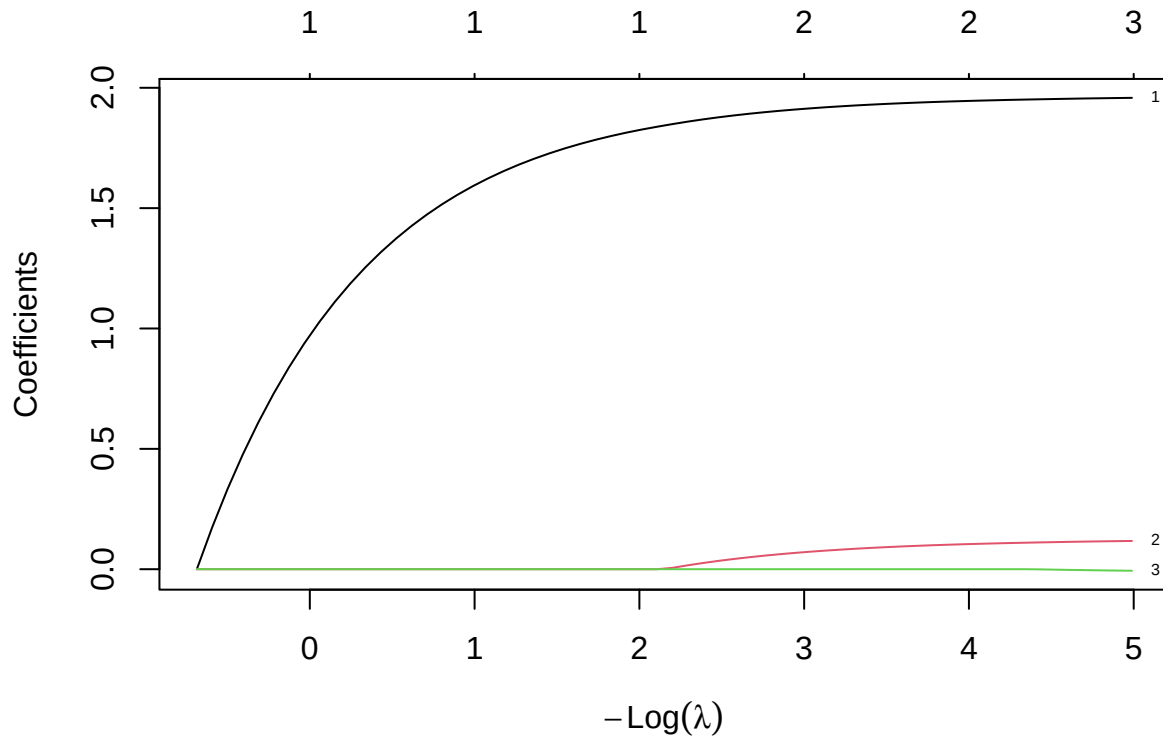
Can set coefficients exactly to zero — performs variable selection.

```
# --- Self-contained: lasso ---
set.seed(2402)
p24_n <- 100
```

```

p24_df <- tibble(
  x1 = rnorm(p24_n), x2 = rnorm(p24_n), x3 = rnorm(p24_n),
  y = 1 + 2 * x1 + rnorm(p24_n)
)
X <- as.matrix(p24_df[, c("x1", "x2", "x3")])
y <- p24_df$y
lasso <- glmnet(X, y, alpha = 1)
plot(lasso, xvar = "lambda", label = TRUE)

```



26.3 24.3 Cross-validated lambda

```

# --- Self-contained: cv.glmnet ---
set.seed(2403)
p24_n <- 100
p24_df <- tibble(
  x1 = rnorm(p24_n), x2 = rnorm(p24_n), x3 = rnorm(p24_n),
  y = 1 + 2 * x1 + rnorm(p24_n)
)
X <- as.matrix(p24_df[, c("x1", "x2", "x3")])
y <- p24_df$y
cv_lasso <- cv.glmnet(X, y, alpha = 1)
coef(cv_lasso, s = "lambda.min")

## 4 x 1 sparse Matrix of class "dgCMatrix"
##          lambda.min

```

```
## (Intercept)  1.10706526
## x1          1.82940244
## x2          .
## x3         -0.03962182
```

lambda.min minimizes CV error; lambda.1se is more parsimonious.

27 Part 25 — Universal Regression Template

Use this for **any** regression question. Replace variable names.

```
# ---- 1. Fit ----
fit <- lm(y ~ x1 + x2 + group + x1:group, data = df)

# ---- 2. Coefficient table ----
kable(tidy(fit, conf.int = TRUE), digits = 4)

# ---- 3. Model fit statistics ----
kable(glance(fit), digits = 4)

# ---- 4. Hypothesis test they asked ----
anova(lm(y ~ x1, data = df), fit) # nested F-test
car::Anova(fit, type = "III")      # Type III SS for unbalanced designs
car::linearHypothesis(fit, "x1 = x2") # linear restriction
AIC(fit1, fit2)                   # non-nested comparison

# ---- 5. Diagnostics ----
par(mfrow = c(2, 2)); plot(fit); par(mfrow = c(1, 1))
shapiro.test(residuals(fit))
bptest(fit)
lmtest::dwtest(fit)
vif(fit) # needs >= 2 predictors
outlierTest(fit)

# ---- 6. Predict ----
newdf <- data.frame(x1 = 1, x2 = 0, group = factor("A"))
predict(fit, newdf, interval = "confidence") # mean response
predict(fit, newdf, interval = "prediction")  # new observation
```

28 Part 26 — Quick Reference Table

If they ask	Use this
Fit model	<code>lm(y ~ x, data = df)</code>
Multiple regression	<code>lm(y ~ x1 + x2, data = df)</code>
Interaction	<code>lm(y ~ x1 * x2, data = df)</code>
Polynomial	<code>lm(y ~ poly(x, 2), data = df)</code>
Categorical predictor	<code>factor(x, levels = ...)</code>
Change reference	<code>relevel(x, ref = "...")</code>
Coefficient table	<code>tidy(fit, conf.int = TRUE)</code>

If they ask	Use this
Model stats	<code>glance(fit)</code>
Full summary	<code>summary(fit)</code>
CI for coefficients	<code>confint(fit)</code>
One-way ANOVA	<code>anova(fit)</code>
Post-hoc Tukey	<code>TukeyHSD(aov(y ~ g))</code>
Post-hoc pairwise	<code>pairwise.t.test(y, g)</code>
ANCOVA	<code>lm(y ~ group + x)</code>
Adjusted means	<code>emmeans(fit, ~ group)</code>
Slopes test	<code>anova(no_int, with_int)</code>
Nested F-test	<code>anova(reduced, full)</code>
Linear restriction	<code>car::linearHypothesis(fit, "x1 = x2")</code>
Type III ANOVA	<code>car::Anova(fit, type = "III")</code>
Non-nested comparison	<code>AIC(fit1, fit2), BIC(...)</code>
Adjusted R ²	<code>summary(fit)\$adj.r.squared</code>
Box-Cox plot	<code>boxCox(fit)</code>
Manual Box-Cox	loop over lambdas
Log transform	<code>lm(log(y) ~ x)</code>
Log-log	<code>lm(log(y) ~ log(x))</code>
Prediction	<code>predict(fit, newdata, interval = ...)</code>
Residual plots	<code>plot(fit)</code>
Normality	<code>shapiro.test(residuals(fit))</code>
Homoskedasticity	<code>bptest(fit)</code>
Autocorrelation	<code>lmtest::dwtest(fit)</code>
Multicollinearity	<code>vif(fit)</code>
Outliers	<code>outlierTest(fit), rstandard(fit)</code>
Influence	<code>cooks.distance(fit)</code>
Logistic	<code>glm(y ~ x, family = binomial)</code>
Odds ratios	<code>exp(coef(fit)), exp(confint(fit))</code>
Poisson	<code>glm(y ~ x, family = poisson)</code>
Negative binomial	<code>MASS::glm.nb(y ~ x)</code>
Piecewise	<code>pmax(time - k, 0)</code>
Backward stepwise	<code>step(fit, direction = "backward")</code>
Forward stepwise	<code>step(null, scope = ..., direction = "forward")</code>
Train/test	<code>sample() + predict()</code>
K-fold CV	loop over folds
LOOCV	<code>residuals(fit)/(1 - hatvalues(fit))</code>
Ridge	<code>glmnet(X, y, alpha = 0)</code>
Lasso	<code>glmnet(X, y, alpha = 1)</code>
CV lambda	<code>cv.glmnet(X, y)</code>

29 Part 27 — Theory Q&A (What to Say)

Q: What is the OLS estimator and why does it minimize SSE? A: $\hat{\beta} = (X^T X)^{-1} X^T y$ is the minimizer of the sum of squared residuals. Taking the derivative of SSE with respect to β and setting it to zero gives the **normal equations** $X^T X \hat{\beta} = X^T y$, which solve to the OLS estimator.

Q: What are the Gauss-Markov assumptions? A: (1) Linearity; (2) errors have mean zero; (3) homoskedasticity; (4) uncorrelated errors; (5) no perfect multicollinearity. Under these, OLS is BLUE. Adding normality gives exact t- and F-tests.

Q: What does the intercept mean? A: The predicted value of y when all predictors are zero. It's only meaningful if $x = 0$ is inside the data range. If not, it's an extrapolation and should not be interpreted.

Q: What's the difference between R^2 and adjusted R^2 ? A: R^2 always increases when you add predictors, even useless ones. Adjusted R^2 penalizes the number of predictors and can decrease. Use adjusted R^2 to compare models with different numbers of predictors.

Q: What's the difference between confidence and prediction intervals? A: A CI is for the mean response at a given x ; a PI is for a single new observation. The PI is wider because it includes the individual error variance σ^2 in addition to the uncertainty about the mean.

Q: What does a significant F-test tell you? A: At least one predictor has a non-zero coefficient. It does not tell you **which** predictors matter. Follow up with t-tests or nested F-tests.

Q: What is multicollinearity and why is it a problem? A: High correlation among predictors inflates the variances of the coefficient estimates, making them unstable and hard to interpret. $VIF > 5$ or 10 indicates a problem. Fixes: drop a predictor, combine them, or use ridge regression.

Q: What is an influential observation? A: An observation whose removal would substantially change the fitted model. Measured by Cook's distance (threshold $4/n$) or DFBETAS. High-leverage points have extreme x values; outliers have extreme y values; influential points are combinations of both.

Q: When would you use WLS instead of OLS? A: When the residual variance is not constant but known (or estimable). WLS weights observations inversely to their variance, giving more weight to more precise observations. It restores BLUE.

Q: What's the difference between AIC and BIC? A: Both penalize complexity. BIC uses a stronger penalty ($k \log n$ vs $2k$) and tends to select simpler models. Use AIC when prediction is the goal; BIC when interpretation/parsimony is the goal.

Q: What does the Box-Cox transformation do? A: It's a family of power transformations $y^{(\lambda)} = (y^\lambda - 1)/\lambda$ for $\lambda \neq 0$ and $\log y$ for $\lambda = 0$. The optimal λ is chosen by maximum likelihood and stabilizes variance, normalizes residuals, and linearizes relationships simultaneously.

Q: How do you interpret a logistic regression coefficient? A: $\hat{\beta}_j$ is the change in **log-odds** per unit increase in x_j . $e^{\hat{\beta}_j}$ is the **odds ratio**: the multiplicative change in the odds of the event per unit increase in x_j .

Q: What is overdispersion and how do you detect it? A: When the variance of the response exceeds the mean (Poisson assumption), usually in count data. Detect by computing Pearson chi-square / df; if > 1.5 , overdispersed. Fixes: quasipoisson (adjusts SEs) or negative binomial (proper likelihood).

Q: What's the difference between ridge and lasso? A: Ridge (L2 penalty) shrinks coefficients toward zero but keeps all predictors. Lasso (L1 penalty) can set coefficients exactly to zero, effectively selecting variables. Elastic net combines both.

Q: Why is stepwise selection controversial? A: It ignores model uncertainty, produces biased p-values and CIs, can overfit, and depends on the arbitrary order of entry. Prefer theory-driven selection, AIC/BIC comparison, or cross-validation.

Q: What is the difference between Type I, II, and III sums of squares? A: Type I: sequential, order matters. Type II: each effect tested after all others except interactions involving it. Type III: each effect tested after all others. Type III is standard for unbalanced designs; use `car::Anova(fit, type = "III")`.

30 Part 28 — Formula Appendix

30.1 Simple linear regression

- $\hat{\beta}_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$
- $\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$
- $\hat{\sigma}^2 = \frac{1}{n-2} \sum e_i^2$
- $\text{SE}(\hat{\beta}_1) = \hat{\sigma} / \sqrt{\sum (x_i - \bar{x})^2}$
- $t = \hat{\beta}_1 / \text{SE}(\hat{\beta}_1) \sim t_{n-2}$

30.2 Multiple regression (matrix form)

- $\hat{\beta} = (X^\top X)^{-1} X^\top y$
- $\text{Var}(\hat{\beta}) = \sigma^2 (X^\top X)^{-1}$
- $\hat{\sigma}^2 = \frac{e^\top e}{n-p-1}$
- $R^2 = 1 - \frac{e^\top e}{\sum (y_i - \bar{y})^2}$
- $F = \frac{(R^2/p)}{(1-R^2)/(n-p-1)} \sim F_{p, n-p-1}$

30.3 Prediction intervals

- Mean response: $\hat{y}_0 \pm t_{\alpha/2, n-p-1} \hat{\sigma} \sqrt{x_0^\top (X^\top X)^{-1} x_0}$
- New observation: $\hat{y}_0 \pm t_{\alpha/2, n-p-1} \hat{\sigma} \sqrt{1 + x_0^\top (X^\top X)^{-1} x_0}$

30.4 Logistic regression

- $\log \frac{p}{1-p} = X\beta$
- $p = \frac{1}{1+e^{-X\beta}}$
- Deviance: $D = -2 \log L$
- Odds ratio: e^{β_j}

30.5 Poisson regression

- $\log \lambda = X\beta$
- $\lambda = e^{X\beta}$
- Mean = variance = λ

30.6 Negative binomial

- $\text{Var}(Y) = \mu + \mu^2/\theta$
- Larger $\theta \rightarrow$ closer to Poisson

30.7 Information criteria

- $AIC = -2\ell + 2k$
- $BIC = -2\ell + k \log n$
- $AIC_c = AIC + \frac{2k(k+1)}{n-k-1}$

30.8 Diagnostic thresholds

- Standardized residual > 3 : outlier
- Cook's D $> 4/n$: influential
- Leverage $> 2(p+1)/n$: high leverage

- VIF > 5: multicollinearity concern
 - VIF > 10: serious multicollinearity
-

31 Part 29 — Common Mistakes

1. **Forgetting to convert categories to factors** — R treats strings as characters and drops them.
 2. **Not setting the reference level** — the alphabetically first level is default, which may not be what you want.
 3. **Interpreting main effects in the presence of interactions** — main effect = effect when the other variable is at its reference / zero.
 4. **Extrapolating beyond the data range** — predicted values outside the observed x range are unreliable.
 5. **Ignoring multicollinearity** — inflated SEs make predictors look insignificant.
 6. **Using R^2 to compare non-nested models** — use AIC/BIC.
 7. **Reporting only point predictions** — always include CIs or PIs.
 8. **Confusing confidence with prediction intervals** — CI is for the mean; PI is for a new observation.
 9. **Not checking diagnostics** — a model with bad residuals is not trustworthy even if R^2 is high.
 10. **Using stepwise selection and reporting p-values as if the model were pre-specified** — they are biased.
 11. **Forgetting family = binomial in logistic** — `glm()` defaults to Gaussian.
 12. **Interpreting Poisson coefficients directly** — they are on the log scale. Exponentiate for rate ratios.
 13. **Ignoring overdispersion in count data** — Poisson SEs will be too small.
 14. **Using `poly(x, 2)` without `raw = TRUE` if you want original-scale coefficients** — otherwise you get orthogonal polynomials.
 15. **Not centering before squaring** — creates multicollinearity with the linear term.
 16. **Comparing models with different response transformations by AIC** — AIC on $\log(y)$ and on y are not comparable.
 17. **Removing outliers just because they are outliers** — only remove if they are data errors or if you report both analyses.
 18. **Forgetting `newdata` in `predict()`** — R uses the original data by default.
 19. **Using `anova()` on non-nested models** — it will produce nonsense.
 20. **Reporting “the model is significant” without saying what that means** — always state the F-statistic, df, and p-value, and interpret.
-

32 Part 30 — The Single Most Important Idea

Every regression question is one of these:

1. **Fit** the model (choose `lm` or `glm`).
2. **Report** coefficients with CIs and p-values.
3. **Test** the hypothesis (t, F, LRT, ANOVA, linear restriction).
4. **Diagnose** (residuals, normality, homoskedasticity, outliers).
5. **Refine** (transform, remove, interact) if needed.
6. **Predict** with the right interval.
7. **Report in words** — equation, interpretation, conclusion.

If you can do these seven steps, you can answer any regression question in the exam.

End of cheat sheet. Good luck in the competition. ““

32.1 What was added and why

Addition	Why it matters
Part 0 — 30-second strategy	Gives an action plan for any question.
Part 1 — Foundations	OLS formula, matrix form, Gauss-Markov, R^2 , F-test — theory questions.
Part 2 — Creating a data frame	Self-contained templates that match the time-series file.
Part 3 — Simple regression with full interpretation	Includes “what the question asks → what you answer” templates.
Part 4 — Multiple regression with holding-others-fixed interpretation	Standard exam phrasing.
Part 5 — Nested F-test / non-nested AIC-BIC	Common exam question “compare these models”.
Part 7 — Interactions with templates	Continuous×continuous, categorical×continuous, plots.
Part 8 — Full ANOVA: one-way, two-way, post-hoc	Includes Tukey, Bonferroni, two-way.
Part 9 — Full ANCOVA with emmeans and slopes test	Exam-favourite topic.
Part 10 — Subset F-test and linear restrictions	<code>linearHypothesis</code> for $\beta_1 = \beta_2$ etc.
Part 11 — Model selection (AIC, BIC, stepwise, all-subsets)	Complete.
Part 12 — Full diagnostics	Residual plots, Shapiro, BP, DW, VIF, Cook’s D, outlier test, all with interpretation.
Part 13 — Transformations and Box-Cox	Grid search, log, log-log (elasticity).
Part 14 — Prediction with clear CI vs PI distinction	Common exam trap.
Part 15 — Polynomial regression	Includes centering and orthogonal polynomials.
Part 16 — Multicollinearity and fixes	VIF, condition number, ridge.
Part 17 — Weighted least squares	Both known and feasible GLS.
Part 18 — Full logistic regression	Log-odds, OR, predicted probabilities, LRT, classification, grouped binomial.
Part 19 — Poisson regression	IRRs, overdispersion, quasipoisson, offsets.
Part 20 — Negative binomial	For overdispersed counts.
Part 21 — Piecewise regression	Continuous and discontinuous versions.
Part 22 — Variable selection	Backward, forward, both, cautions.
Part 23 — Train/test, K-fold, LOOCV, bootstrap	Full evaluation toolkit.
Part 24 — Ridge and lasso	Modern regularization.
Part 25 — Universal template	Ready to copy-paste.
Part 26 — Quick reference table	One-liner answers.
Part 27 — Theory Q&A	Ready-made answers for conceptual questions.
Part 28 — Formula appendix	All formulas in one place.
Part 29 — Common mistakes	20 traps to avoid.
Part 30 — Single most important idea	The seven-step pipeline.